ПРИМЕР

Вызов: SecurityManagerGroup.GroupEffectiveRights("Управление состоянием оборудования")

Результат запроса списка приложений в JSON-формате:

```
{
 "data": [
  {
   "appName": "Управление состоянием оборудования",
   "tokenType": "bool",
   "tokenName": "Управление резервуаром",
   "tokenDescription": "Право управления резервуаром",
   "ownerId": "alpha:07dba445-1903-498c-abdf-af7f55c145c4",
   "tokenValueAllow": "TRUE",
   "tokenValueDeny": "FALSE"
  }
 ]
}
```

New

```
void New()
```

Освобождает элемент от загруженной в него информации для создания новой группы.

Функция не требует входных параметров.

ПРИМЕР

Вызов: SecurityManagerGroup.New()

ПРИМЕЧАНИЕ

После создания новой группы не забудьте загрузить её в подсистему безопасности с помощью функции Save().

AddRole

```
void AddRole(string roleUID)
```

Назначает указанную роль группе, загруженной в элемент.

Входной параметр - уникальный идентификатор (uid) роли вида alpha:xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx (тип значения - string).

ПРИМЕР

Вызов: SecurityManagerGroup.AddRole("alpha:4ce9483e-e137-41c2-bf1c-343e9b731f20")

DeleteRole

```
void DeleteRole(string roleUID)
```

Лишает указанной роли группу, загруженную в элемент.

Входной параметр - уникальный идентификатор (uid) роли вида alpha:xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx (тип значения - string).

ПРИМЕР

Вызов: SecurityManagerGroup.DeleteRole("alpha:4ce9483e-e137-41c2-bf1c-343e9b731f20")

AddRight

```
void AddRight(string appID, string rightName, uint1 rightType, string allowedValue, string
forbidenValue)
```

Назначает право группе, загруженной в элемент.

Входные параметры:

- ID приложения, содержащего право (тип значения - string);
- имя права (тип значения - string);
- тип права (тип значения - uint1):
  - **«0»** - строковый;
  - **«1»** - логический;
- разрешающее значение права (тип значения - string);
- запрещающее значение права (тип значения - string).

Для строковых прав значения (если их несколько) можно разделить с помощью сочетания символов \n. Пример: "значение1\nзначение2\nзначение3".

ПРИМЕР

Вызов: SecurityManagerGroup.AddRight("Управление состоянием оборудования","Управление насосом",1,"true","false"

ChangeRight

```
void ChangeRight(string appID, string rightName, string allowedValue, string forbidenValue)
```

Меняет значения прав группы, загруженной в элемент.

Входные параметры:

- ID приложения, содержащего право (тип значения - string);
- имя права (тип значения - string);

- новое разрешающее значение права (тип значения - string);
- новое запрещающее значение права (тип значения - string).

Для строковых прав значения (если их несколько) можно разделить с помощью сочетания символов \n. Пример: "значение1\nзначение2\nзначение3".

ПРИМЕР

Вызов: SecurityManagerGroup.ChangeRight("Управление состоянием оборудования","Управление насосом","false","true")

DeleteRight

```
void DeleteRight(string appID, string rightName)
```

Лишает права группу, загруженную в элемент.

Входные параметры:

- ID приложения, содержащего право (тип значения - string);
- имя права (тип значения - string).

ПРИМЕР

Вызов: SecurityManagerGroup.DeleteRight("Управление состоянием оборудования","Управление насосом")

GetErrorDescriptionByCode

```
string GetErrorDescriptionByCode(uint1 FailReasonCode)
```

Возвращает текстовое описание ошибок, возникающих при загрузке и сохранении групп.

Входной параметр - переменная FailReasonCode (тип значения - uint1).

2.9.3. События

LoadComplete

Информация о группе загружена в элемент.

Активируется в случае успешного завершения операции Load() (стр. 133).

SaveComplete

Информация о группе сохранена в подсистему безопасности Alpha.Security.

Активируется в случае успешного завершения операции Save() (стр. 133).

LoadFailed

Информация о группе не загружена в элемент.

Активируется в случае неуспешного завершения операции Load() (стр. 133).

Возвращает номер ошибки в переменную FailReasonCode, тип значения - uint1.

ПРИМЕЧАНИЕ

Получить текст ошибки можно, вызвав функцию GetErrorDescriptionByCode() (стр. 133).

SaveFailed

Информация о группе не сохранена в подсистему безопасности Alpha.Security.

Активируется в случае неуспешного завершения операции Save() (стр. 133).

Возвращает номер ошибки в переменную FailReasonCode, тип значения - uint1.

ПРИМЕЧАНИЕ

Получить текст ошибки можно, вызвав функцию GetErrorDescriptionByCode() (стр. 133).

2.10. Мастер конфигурирования Security (Configurator)

Компонент предназначен для чтения и обновления конфигурации службы Агент Alpha.Security.

Обычно сразу после установки подсистемы безопасности Alpha.Security необходимо настроить ее компоненты - в том числе, службу Агент Alpha.Security. Настройка службы выполняется в конфигурационном файле `alpha.security.agent.xml`, расположенном в `C:\Program Files\Automiq\Alpha.Security` (для Windows) или в `/opt/Automiq/Alpha.Security` (для Linux). Подробнее настройка службы описана в документе на Alpha.Security, в разделе **«Агент безопасности и его настройка»**. Вносить изменения в конфигурационный файл можно вручную, либо из проекта Alpha.HMI с помощью свойств, функций и событий экземпляра **Мастер конфигурирования Security**.

Компонент следует использовать следующим образом:

- Для чтения конфигурации необходимо вызвать функцию Read() (стр. 146). Она поместит данные из конфигурационного файла в компонент. После этого можно будет обратиться к данным, прочитав значения свойств компонента или вызвав функции, название которых начинается с **Get** (**GetLdapHost()**, **GetLdapPort()** и пр.).
- Для создания новой конфигурации необходимо сначала указать новые данные, поместив их в значения свойств и во входные параметры функций, название которых начинается с **Add** (**AddLdap()**, **AddLogConsumer()** и пр.). После этого следует вызвать функцию Generate() (стр. 145), которая создаст текст нового конфигурационного файла.

Пример использования компонента в проекте Alpha.HMI приведен в разделе 1.7. Конфигурирование агента безопасности (стр. 40).

2.10.1. Свойства

Адрес Net агента (NetHost)

IP-адрес или имя компьютера, на котором установлен Net-агент, к которому подключается агент безопасности. Под Net-агентом подразумевается служба **`Alpha.Net.Agent`** (на ОС Windows) или **`Alpha.Net.service`** (на ОС Linux). Тип значения - string.

Зачастую Net-агент установлен на локальном компьютере.

Значение свойства соответствует значению атрибута `Address` элемента `<EntryPointNetAgent>` в конфигурационном файле агента безопасности.

Порт Net агента (NetPort)

Номер порта для подключения к Net-агенту, к которому подключается агент безопасности. Тип значения - int4.

Значение свойства соответствует значению атрибута `Port` элемента `<EntryPointNetAgent>` в конфигурационном файле агента безопасности.

Адрес LDAP сервера (LdapHost)

IP-адрес или имя компьютера, на котором развернут LDAP-сервер, к которому подключается агент безопасности. Тип значения - string.

Свойство устарело. Теперь при настройке агента может быть указано несколько LDAP-серверов, поэтому значения адресов записываются в компонент в виде массива.

Чтобы прочитать конкретное значение в массиве:

1. Вызовите функцию Read() (стр. 146), предназначенную для записи текущей конфигурации агента в массив.
2. Вызовите функцию GetLdapHost() (стр. 147), указав в качестве входного параметра номер элемента в массиве.

Количество элементов в массиве (LDAP-серверов) записывается в значение свойства LdapCount (стр. 145). Каждый элемент массива соответствует значению атрибута `Address` одного из элементов `<LDAPServer>` в конфигурационном файле агента безопасности.

Чтобы добавить описание нового LDAP-сервера, используйте функцию AddLdap() (стр. 146)

Порт LDAP сервера (LdapPort)

Номер порта для подключения к LDAP-серверу. Тип значения - string.

Свойство устарело. Теперь при настройке агента может быть указано несколько LDAP-серверов, поэтому значения портов записываются в компонент в виде массива.

Чтобы прочитать конкретное значение в массиве:

1. Вызовите функцию Read() (стр. 146), предназначенную для записи текущей конфигурации агента в массив.

2. Вызовите функцию GetLdapPort() (стр. 147), указав в качестве входного параметра номер элемента в массиве.

Количество элементов в массиве (LDAP-серверов) записывается в значение свойства LdapCount (стр. 145). Каждый элемент массива соответствует значению атрибута `Port` одного из элементов `<LDAPServer>` в конфигурационном файле агента безопасности.

Чтобы добавить описание нового LDAP-сервера, используйте функцию AddLdap() (стр. 146)

Использование защищенного соединения (UseSecureConnection)

Указывает агенту безопасности необходимость использования защищенного соединения с LDAP-сервером. Тип значения - bool.

Значение свойства соответствует значению атрибута `value` элемента `<LdapSecure>` в конфигурационном файле агента безопасности.

Режим работы контроля целостности (ICMode)

Указывает, включен ли режим контроля целостности файлов и папок. Тип значения - bool.

Значение свойства соответствует значению атрибута `ICMode` элемента `<Options>` в конфигурационном файле агента безопасности:

- **«0»** - контроль целостности отключен, значение свойства - **«fasle»**;
- **«1»** - контроль целостности включен, значение свойства - **«true»**.

Пользователь LDAP (LdapUser)

Учетная запись администратора LDAP-сервера. Тип значения - string.

Поскольку учетная запись хранится на LDAP-сервере в виде каталога, обращение к ней происходит в стандартном для LDAP виде:

```
cn=ЛогинПользователя,dc=ДоменLDAPсервера
```

Значение свойства соответствует значению атрибута `value` элемента `<LdapUser>` в конфигурационном файле агента безопасности.

ПРИМЕР

Пример значения: **«cn=Manager,dc=maxcrc,dc=com»**

Пароль пользователя LDAP (LdapUserPass)

Пароль учетной записи администратора LDAP-сервера. Тип значения - string.

Поскольку пароли являются секретной информацией, просмотреть текущее значение при чтении конфигурации нельзя. При создании новой конфигурации свойство используется для записи нового значения. Значение пароля, введенное в открытом виде, будет зашифровано.

Адрес папки системы безопасности (LdapDN)

Название корневого каталога на LDAP-сервере. Тип значения - string.

Обращение к каталогу происходит в стандартном для LDAP виде:

```
ou=НазваниеПапки,dc=ДоменLDAPсервера
```

Значение свойства соответствует значению атрибута `value` элемента `<SecurityDn>` в конфигурационном файле агента безопасности.

ПРИМЕР

Пример значения: **«cn=ou=AlphaSecurity,dc=maxcrc,dc=com»**

Имя гостевой УЗ (GuestName)

Имя гостевой учетной записи - записи, чьи права используются, когда нет активной пользовательской сессии. Тип значения - string.

Значение свойства соответствует значению атрибута `value` элемента `<GuestDisplayName>` в конфигурационном файле агента безопасности.

Пользователь по умолчанию (DefaultUser)

Имя учетной записи пользователя по умолчанию - пользователя, чья сессия становится активной, если другие пользователи не авторизовались в подсистеме. Тип значения - string.

Значение свойства соответствует значению атрибута `value` элемента `<DefaultUser>` в конфигурационном файле агента безопасности.

Пароль пользователя по умолчанию (DefaultUserPass)

Пароль пользователя по умолчанию. Тип значения - string.

Значение свойства соответствует значению атрибута `value` элемента `<DefaultUserPassword>` в конфигурационном файле агента безопасности.

Уровень логирования (LoggerLevel)

Означает количество информации, выводимой в лог подсистемы безопасности. Тип значения - int4.

Значение свойства соответствует значению атрибута `LoggerLevel` элемента `<Options>` в конфигурационном файле агента безопасности:

- **«0»** - в лог выводится минимум информации;
- **«2»** - в лог выводится вся основная информация о работе Alpha.Security;
- **«5»** - в лог выводится дополнительная информация о работе Alpha.Security помимо основной.

Комбинации блокировки (DriverString)

Перечень клавиш, заблокированных для использования пользователем. Тип значения - string.

Заблокированные сочетания указаны в виде SCAN-кодов клавиш, разделенных внутри сочетания символом **«+»**, а между сочетаниями - символом **«;»**.

Значение свойства соответствует значению атрибута `kbDriverString` элемента `<Options>` в конфигурационном файле агента безопасности.

ПРИМЕР

Пример значения: **«0x1D+0x38+0x53;0x1D+0x2A+0x01;»**

В данном примере заблокированы сочетания **«ctrl+alt+del»** и **«ctrl+shift+esc»**.

Трассировка аудита (TraceAudit)

Включает трансляцию сообщений аудита в системный журнал. Тип значения - bool.

Значение свойства соответствует значению атрибута `TraceAudit` элемента `<AuditLogConsumers>` в конфигурационном файле агента безопасности:

- **«0»** - сообщения не транслируются в системный журнал, значение свойства - **«fasle»**;
- **«1»** - сообщения транслируются в системный журнал, значение свойства - **«true»**.

Кэш прав пользователя (UseRightsStorageCache)

Указывает, используются ли кэшированные значения прав пользователя. Тип значения - bool.

Значение свойства соответствует значению атрибута `UseRightsCacheStorage` элемента `<Options>` в конфигурационном файле агента безопасности:

- **«0»** - актуальные значения прав пользователя запрашиваются с LDAP-сервера постоянно, значение свойства - **«false»**;
- **«1»** - значения прав пользователя запрашиваются с LDAP-сервера только в момент входа пользователя в подсистему, а в процессе его работы значения запрашиваются из кэша. Значение свойства в таком случае- **«true»**.

Префикс сообщений аудита (MesPrefix)

Префикс сообщений аудита. Тип значения - string.

Значение свойства соответствует значению атрибута `value` элемента `<mesPrefix>` в конфигурационном файле агента безопасности.

Ошибка конфигурирования (Error)

Текст ошибки, возникшей при попытке создания новой конфигурации Агент Alpha.Security с помощью функции Generate() (стр. 145). Тип значения - string.

Только для чтения в режиме рантайма.

Сгенерированная строка (GeneratedString)

Полный текст конфигурационного файла Агент Alpha.Security в xml-формате, созданного в результате вызова функции Generate() (стр. 145). Тип значения - string.

Только для чтения в режиме рантайма.

Ошибка чтения (ReadError)

Текст ошибки, возникшей при попытке чтения конфигурации Агент Alpha.Security с помощью функции Read () (стр. 146). Тип значения - string.

Только для чтения в режиме рантайма.

Количество LDAP серверов (LdapCount)

Количество LDAP-серверов, указанных в конфигурационном файле Агент Alpha.Security. Тип значения - int4.

Только для чтения в режиме рантайма.

Количество потребителей аудита (ConsumersCount)

Количество серверов-потребителей аудита, указанных в конфигурационном файле Агент Alpha.Security. Тип значения - int4.

Только для чтения в режиме рантайма.

Незаполненные поля (UndefinedListItems)

Список полей (свойств), для которых обязательно указать значение для создания новой конфигурации Агент Alpha.Security с помощью функции Generate() (стр. 145). Тип значения - string.

Только для чтения в режиме рантайма.

2.10.2. Функции

Generate

```
int4 Generate(bool activationFlag)
```

Создает текст конфигурационного файла Агент Alpha.Security.

Входной параметр отвечает за совместимость создаваемой конфигурации со старыми версиями Alpha.Security (тип значения - bool). В качестве значения следует указывать **«true»**.

Результат:

- сразу после вызова функции активируется событие GenerationStarted (стр. 157);
- в случае успешного завершения операции активируется событие GenerationFinished (стр. 157), а строка-результат записывается в свойство GeneratedString (стр. 145);
- в случае неуспешного завершения операции активируется событие GenerationFailure (стр. 157).

Для создания конфигурации должны быть заполнены все обязательные поля. Обязательно следует указать:

- адрес и порт Net-агента - в соответствующих свойствах (стр. 141);
- параметры хотя бы одного LDAP-сервера - с помощью функции AddLdap() (стр. 146);
- учетную запись администратора LDAP (в стандартном для LDAP виде) и пароль (стр. 142);
- пароль пользователя, чья учетная запись используется по умолчанию (стр. 143);
- адрес корневой папки LDAP-сервера (стр. 143);
- защищено ли соединение агента безопасности с LDAP-сервером (стр. 142).

Если в процессе создания конфигурации возникнет ошибка, функция вернет ее код. Чтобы ознакомиться с текстом ошибки, обратитесь ко значению свойства Ошибка конфигурирования (Error) (стр. 144) компонента, либо ко значению внутренней переменной Error события GenerationFailure (стр. 157).

Read

```
void Read(string XML)
```

Выполняет чтение конфигурации из конфигурационного файла Агент Alpha.Security.

Входной параметр - строка, содержащая полный текст конфигурационного файла Агент Alpha.Security в xml-формате (тип значения - string).

Чтобы получить такую строку из файла с помощью компонентов Alpha.HMI, используйте функцию ReadTextFile() компонента **Окружение: файлы (File System)**. В качестве входного параметра функции укажите полный путь к файлу конфигурации.

ПРИМЕР

Пример вызова:

```
Configurator.Read(FileSystem.ReadTextFile("C:/Program
Files/Automiq/Alpha.Security/alpha.security.agent.xml"));
```

Результат:

- сразу после вызова функции активируется событие ReadingStarted (стр. 157);
- в случае успешного завершения операции активируется событие ReadingFinished (стр. 157);
- в случае неуспешного завершения операции активируется событие ReadingFailure (стр. 157).

AddLdap

```
bool AddLdap(string LDAPHost, int4 LDAPPort)
```

Добавляет описание LDAP-сервера, к которому должен подключаться агент безопасности. Добавленное описание хранится во внутреннем массиве компонента.

Входные параметры:

- IP-адрес или имя компьютера, на котором развернут LDAP-сервер;
- номер порта для подключения к LDAP-серверу.

Функция возвращает результат добавления LDAP-сервера в конфигурацию:

- **«true»** - информация добавлена;
- **«false»** - информацию не удалось добавить.

В случае успешного завершения операции активируется событие LdapListChanged (стр. 157).

ClearLdapList

```
void ClearLdapList()
```

Очищает внутренний массив LDAP-серверов компонента.

Функция не требует входных параметров.

В случае успешного завершения операции активируется событие LdapListChanged (стр. 157).

GetLdapHost

```
string GetLdapHost(int4 index)
```

Предоставляет IP-адрес одного из указанных в конфигурационном файле LDAP-серверов.

Поскольку в конфигурационном файле может быть указано несколько LDAP-серверов, перечень их IP-адресов записывается в компонент в виде массива. Обратиться к конкретному значению можно по номеру его записи в массив.

Каждый элемент массива соответствует значению атрибута `Address` одного из xml-элементов `<LDAPServer>`, вложенного в xml-элемент `<LdapHosts>`, в конфигурационном файле агента безопасности.

GetLdapPort

```
int4 GetLdapPort(int4 index)
```

Предоставляет номер порта для подключения к одному из указанных в конфигурационном файле LDAP-серверов.

Поскольку в конфигурационном файле может быть указано несколько LDAP-серверов, перечень портов для подключения к ним записывается в компонент в виде массива. Обратиться к конкретному значению можно по номеру его записи в массив.

Каждый элемент массива соответствует значению атрибута `Port` одного из xml-элементов `<LDAPServer>`, вложенного в xml-элемент `<LdapHosts>`, в конфигурационном файле агента безопасности.

GetAuditServerHost

```
string GetAuditServerHost(int4 index)
```

Предоставляет IP-адрес одного из указанных в конфигурационном файле серверов-потребителей аудита.

Поскольку в конфигурационном файле может быть указано несколько серверов-потребителей, перечень их IP-адресов записывается в компонент в виде массива. Обратиться к конкретному значению можно по номеру его записи в массив.

Каждый элемент массива соответствует значению атрибута `Host` xml-элемента `<Server>`, вложенного в один из xml-элементов `<OpcDaLogConsumer>`, в конфигурационном файле агента безопасности.

GetAuditServerPort

```
int4 GetAuditServerPort(int4 index)
```

Предоставляет порт для подключения к одному из указанных в конфигурационном файле серверов-потребителей аудита.

Поскольку в конфигурационном файле может быть указано несколько серверов-потребителей, перечень портов для подключения к ним записывается в компонент в виде массива. Обратиться к конкретному значению можно по номеру его записи в массив.

Каждый элемент массива соответствует значению атрибута `TCPServerPort` xml-элемента `<Server>`, вложенного в один из xml-элементов `<OpcDaLogConsumer>`, в конфигурационном файле агента безопасности.

GetServerProgId

```
string GetAuditServerPort(int4 index)
```

Предоставляет ProgID (строковый идентификатор) одного из указанных в конфигурационном файле серверов-потребителей аудита.

Поскольку в конфигурационном файле может быть указано несколько серверов-потребителей, перечень их ProgID записывается в компонент в виде массива. Обратиться к конкретному значению можно по номеру его записи в массив.

Каждый элемент массива соответствует значению атрибута `ProgId` xml-элемента `<Server>`, вложенного в один из xml-элементов `<OpcDaLogConsumer>`, в конфигурационном файле агента безопасности.

GetServerType

```
string GetServerType(int4 index)
```

Предоставляет информацию о типе одного из указанных в конфигурационном файле серверов-потребителей аудита.

Поскольку в конфигурационном файле может быть указано несколько серверов-потребителей, перечень их типов записывается в компонент в виде массива. Обратиться к конкретному значению можно по номеру его записи в массив.

Каждый элемент массива соответствует значению атрибута `Type` xml-элемента `<Server>`, вложенного в один из xml-элементов `<OpcDaLogConsumer>`, в конфигурационном файле агента безопасности.

GetSeverityCount

```
int4 GetServerType(int4 index)
```

Предоставляет количество категорий важности сообщений для одного из указанных в конфигурационном файле серверов-потребителей аудита.

Поскольку в конфигурационном файле может быть указано несколько серверов-потребителей, значение от каждого из них записывается в компонент в виде элемента массива. Обратиться к конкретному значению можно по номеру его записи в массив.

Каждый элемент массива соответствует количеству элементов `<Severity>` внутри элемента `<SeverityMap>`, вложенного в один из элементов `<OpcDaLogConsumer>`, в конфигурационном файле агента безопасности.

GetSignalsCount

```
int4 GetSignalsCount(int4 index)
```

Предоставляет количество сигналов, предназначенных для записи сообщений аудита, в одном из указанных в конфигурационном файле серверов-потребителей аудита.

Поскольку в конфигурационном файле может быть указано несколько серверов-потребителей, значение для каждого из них записывается в компонент в виде элемента массива. Обратиться к конкретному значению можно по номеру его записи в массив.

Каждый элемент массива соответствует количеству элементов `<Signal>` внутри элемента `<SignalMap>`, вложенного в один из элементов `<OpcDaLogConsumer>`, в конфигурационном файле агента безопасности.

GetSeverityCategory

```
string GetSeverityCategory(int4 i, int4 k)
```

Предоставляет название выбранной категории важности сообщений.

Поскольку в конфигурационном файле может быть указано несколько серверов-потребителей, каждый из них описывается в компоненте в виде элемента массива A[i]. Каждый элемент массива A[i] представляет собой массив B, каждый элемент B[i,k] которого описывает одну из категорий важности описываемого сервера A[i].

```
Массив A, описывающий сервера-потребители = [[Массив B, описывающий первый сервер], [Массив
B, описывающий второй сервер]]
Массив B, описывающий первый сервер = [[Описание первой категории важности], [Описание
второй категории важности]]
Массив B, описывающий второй сервер = [[Описание первой категории важности], [Описание
второй категории важности]]
```

Обратиться к конкретной категории важности можно по номеру элемента в массивах A и B (i, k).

Каждый элемент массива B[i,k] соответствует одному из значений атрибута `Category` xml-элемента `<Severity>`, вложенного в xml-элемент `<SeverityMap>` одного из xml-элементов `<OpcDaLogConsumer>` в конфигурационном файле агента безопасности.

Пример использования приведен в описании функции GetSeverityValue().

GetSeverityValue

```
int4 GetSeverityValue(int4 i, int4 k)
```

Предоставляет числовое значение выбранной категории важности сообщений.

Поскольку в конфигурационном файле может быть указано несколько серверов-потребителей, каждый из них описывается в компоненте в виде элемента массива A[i]. Каждый элемент массива A[i] представляет собой массив B, каждый элемент B[i,k] которого описывает одну из категорий важности описываемого сервера A[i].

```
Массив A, описывающий сервера-потребители = [[Массив B, описывающий первый сервер], [Массив
B, описывающий второй сервер]]
Массив B, описывающий первый сервер = [[Описание первой категории важности], [Описание
второй категории важности]]
Массив B, описывающий второй сервер = [[Описание первой категории важности], [Описание
второй категории важности]]
```

Обратиться к конкретной категории важности можно по номеру элемента в массивах A и B (i, k).

Каждый элемент массива B[i,k] соответствует одному из значений атрибута `Value` xml-элемента `<Severity>`, вложенного в xml-элемент `<SeverityMap>` одного из xml-элементов `<OpcDaLogConsumer>` в конфигурационном файле агента безопасности.

ПРИМЕР

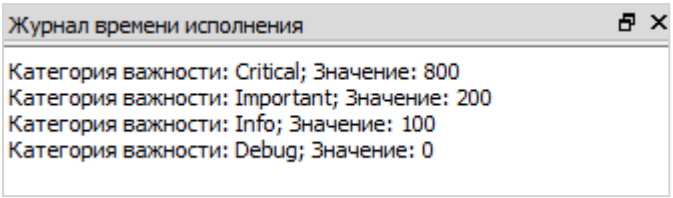
Допустим, в конфигурационном файле описан один сервер-потребитель сообщений (один элемент `<OpcDaLogConsumer>`), для которого описано четыре категории важности сообщений аудита:

```
<AuditLogConsumers TraceAudit="1">
 <OpcDaLogConsumer>
  <Server Host="127.0.0.1" Type="OPC" ...>
   <SeverityMap>
    <Severity Category="Critical" Value="800"/>
    <Severity Category="Important" Value="200"/>
    <Severity Category="Info" Value="100"/>
    <Severity Category="Debug" Value="0"/>
   </SeverityMap>
  </Server>
 </OpcDaLogConsumer>
</AuditLogConsumers>
```

Чтобы получить список категорий важности сообщений с их значениями в каждом сервере-потребителе, вызовите нужные функции в коде, выполняющемся в случае успешного чтения конфигурации Агент Alpha.Security (например, в обработчике события ReadingFinished (стр. 157)). Укажите в качестве входных параметров индексы **«i»** (индекс в массиве серверов-потребителей) и **«k»** (индекс в массиве категорий важности). Приведенный ниже пример написан на языке Alpha.Om, в нем итоговый список записывается в лог:

```
i: int4 = 0;
while (i < Configurator.ConsumersCount) //цикл выполняется, пока в массиве
A
не будут описаны все сервера-потребители
{
  k: int4 = 0;
  while (k < Configurator.GetSeverityCount(i)) //цикл выполняется, пока в
массив B
  не будут записаны все категории важности i-го сервера-потребителя
  {
    DebugTool.Log("Категория важности: "+Configurator.GetSeverityCategory
(i,k)+";
    Значение: "+String.ToString(Configurator.GetSeverityValue(i,k)));
    //в Журнал времени исполнения записываются названия и значения
категорий
    важности для i-го сервера
    k += 1;
  }
  i += 1;
}
```

В результате вызова функций в **Журнал времени исполнения** запишутся названия категорий важности с их значениями:



GetSignalName

```
string GetSignalName(int4 i, int4 k)
```

Предоставляет название указанного сигнала, предназначенного для записи сообщений аудита, в указанном сервере-потребителе.

Поскольку в конфигурационном файле может быть указано несколько серверов-потребителей, каждый из них описывается в компоненте в виде элемента массива A[i]. Каждый элемент массива A[i] представляет собой массив B, каждый элемент B[i,k] которого описывает один из сигналов в описываемом сервере A[i].

```
Массив A, описывающий сервера-потребители = [[Массив B, описывающий первый сервер], [Массив
B, описывающий второй сервер]]
Массив B, описывающий первый сервер = [[Описание первого сигнала], [Описание второго
сигнала]]
Массив B, описывающий второй сервер = [[Описание первого сигнала], [Описание второго
сигнала]]
```

Обратиться к конкретному сигналу можно по номеру элемента в массивах A и B (i, k).

Каждый элемент массива B[i,k] соответствует одному из значений атрибута `Name` xml-элемента `<Signal>`, вложенного в xml-элемент `<SignalMap>` одного из xml-элементов `<OpcDaLogConsumer>` в конфигурационном файле агента безопасности.

Пример использования приведен в описании функции GetSignalType().

GetSignalMode

```
int4 GetSignalMode(int4 i, int4 k)
```

Предоставляет информацию о режиме записи сообщения в указанный сигнал в указанном сервере-потребителе. Возвращает значение в числовом виде, где:

- **«1»** - соответствует **«DynamicEvent»** (запись сообщения с xml-конструкцией, описывающей динамическое событие);
- **«2»** - соответствует **«Value»** (обычная запись сообщения).

Поскольку в конфигурационном файле может быть указано несколько серверов-потребителей, каждый из них описывается в компоненте в виде элемента массива A[i]. Каждый элемент массива A[i] представляет собой массив B, каждый элемент B[i,k] которого описывает один из сигналов в описываемом сервере A[i].

```
Массив A, описывающий сервера-потребители = [[Массив B, описывающий первый сервер], [Массив
B, описывающий второй сервер]]
Массив B, описывающий первый сервер = [[Описание первого сигнала], [Описание второго
сигнала]]
Массив B, описывающий второй сервер = [[Описание первого сигнала], [Описание второго
сигнала]]
```

Обратиться к конкретному сигналу можно по номеру элемента в массивах A и B (i, k).

Каждый элемент массива B[i,k] соответствует одному из значений атрибута `Mode` xml-элемента `<Signal>`, вложенного в xml-элемент `<SignalMap>` одного из xml-элементов `<OpcDaLogConsumer>` в конфигурационном файле агента безопасности.

Пример использования приведен в описании функции GetSignalType().

GetSignalType

```
string GetSignalType(int4 i, int4 k)
```

Предоставляет название типа сообщений в указанном сигнале для указанного сервера-потребителя.

Поскольку в конфигурационном файле может быть указано несколько серверов-потребителей, каждый из них описывается в компоненте в виде элемента массива A[i]. Каждый элемент массива A[i] представляет собой массив B, каждый элемент B[i,k] которого описывает один из сигналов в описываемом сервере A[i].

```
Массив A, описывающий сервера-потребители = [[Массив B, описывающий первый сервер], [Массив
B, описывающий второй сервер]]
Массив B, описывающий первый сервер = [[Описание первого сигнала], [Описание второго
сигнала]]
Массив B, описывающий второй сервер = [[Описание первого сигнала], [Описание второго
сигнала]]
```

Обратиться к конкретному сигналу можно по номеру элемента в массивах A и B (i, k).

Каждый элемент массива B[i,k] соответствует одному из значений атрибута `Type` xml-элемента `<Signal>`, вложенного в xml-элемент `<SignalMap>` одного из xml-элементов `<OpcDaLogConsumer>` в конфигурационном файле агента безопасности.

ПРИМЕР

Допустим, в конфигурационном файле описан один сервер-потребитель сообщений (один элемент `<OpcDaLogConsumer>`), в котором для записи сообщений аудита предназначено несколько сигналов:

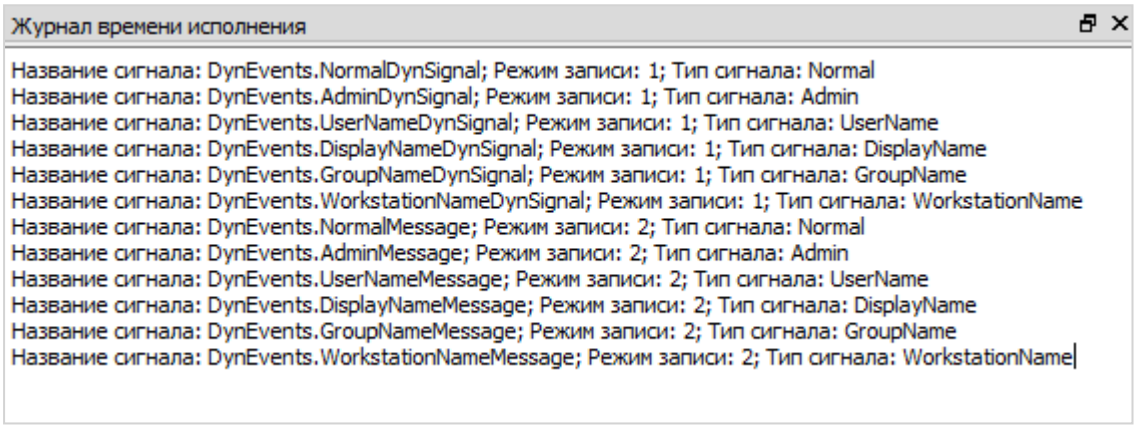
```
<AuditLogConsumers TraceAudit="1">
 <OpcDaLogConsumer>
  <Server Host="127.0.0.1" Type="OPC" ...>
   <SignalMap>
    <Signal Name="DynEvents.NormalDynSignal" Mode="DynamicEvent"
Type="Normal"/>
    <Signal Name="DynEvents.AdminDynSignal" Mode="DynamicEvent"
Type="Admin"/>
    <Signal Name="DynEvents.UserNameDynSignal" Mode="DynamicEvent"
Type="UserName"/>
    <Signal Name="DynEvents.DisplayNameDynSignal" Mode="DynamicEvent"
Type="DisplayName"/>
    <Signal Name="DynEvents.GroupNameDynSignal" Mode="DynamicEvent"
Type="GroupName"/>
    <Signal Name="DynEvents.WorkstationNameDynSignal" Mode="DynamicEvent"
Type="WorkstationName"/>
    <Signal Name="DynEvents.NormalMessage" Mode="Value" Type="Normal"/>
    <Signal Name="DynEvents.AdminMessage" Mode="Value" Type="Admin"/>
    <Signal Name="DynEvents.UserNameMessage" Mode="Value"
Type="UserName"/>
    <Signal Name="DynEvents.DisplayNameMessage" Mode="Value"
Type="DisplayName"/>
    <Signal Name="DynEvents.GroupNameMessage" Mode="Value"
Type="GroupName"/>
    <Signal Name="DynEvents.WorkstationNameMessage" Mode="Value"
Type="WorkstationName"/>
   </SignalMap>
  </Server>
 </OpcDaLogConsumer>
</AuditLogConsumers>
```

Чтобы получить список параметров сигналов в каждом сервере-потребителе, вызовите нужные функции в коде, выполняющемся в случае успешного чтения конфигурации Агент Alpha.Security (например, в обработчике события ReadingFinished (стр. 157)). Укажите в качестве входных параметров индексы «**i**» (индекс в массиве серверов-потребителей) и «**k**» (индекс в массиве сигналов). Приведенный ниже пример написан на языке Alpha.Om, в нем список сигналов с их параметрами записывается в лог:

```
i: int4 = 0;
while (i < Configurator.ConsumersCount) //цикл выполняется, пока в массиве
A не будут описаны все сервера-потребители
{
  k: int4 = 0;
  while (k < Configurator.GetSignalsCount(i)) //цикл выполняется, пока в
массив B не будут записаны все сигналы i-го сервера-потребителя
```

```
  {
    DebugTool.Log("Название сигнала: "+Configurator.GetSignalName(i,k)+";
Режим записи: "+String.ToString(Configurator.GetSignalMode(i,k))+"; Тип
сообщения: "+Configurator.GetSignalType(i,k)); //в Журнал времени
исполнения записывается список сигналов i-го сервера
    k += 1;
  }
  i += 1;
}
```

В результате вызова функций в **Журнал времени исполнения** запишется список сигналов с их параметрами:



ClearLogConsumersList

```
void ClearLogConsumersList()
```

Очищает внутренний массив серверов-потребителей в компоненте.

Функция не требует входных параметров.

В случае успешного завершения операции активируется событие ConsumersListChanged (стр. 158).

AddLogConsumer

```
void AddLogConsumer(string Host, string HostTcpReserve, int4 TCPServerPort, string ProgID,
string Type)
```

Добавляет описание сервера-потребителя аудита сообщений. Добавленное описание хранится во внутреннем массиве компонента.

Входные параметры:

- имя или IP-адрес сервера;
- имя или IP-адрес сервера для резервного канала связи с сервером TCP (значение может быть пустым);

- порт для подключения к TCP-серверу (если выбран TCP-сервер);
- строковый идентификатор OPC-сервера (если выбран OPC-сервер);
- тип сервера:
  - для Windows возможны значения **«OPC»** (по умолчанию) и **«TCP»**;
  - для других ОС допустимо только **«TCP»**.

В случае успешного завершения операции активируется событие ConsumersListChanged (стр. 158).

AddSeverity

```
bool AddSeverity(int4 consumerIndex, string severityCategory, int4 severityValue)
```

Добавляет в компонент категорию важности сообщений для указанного сервера-потребителя.

Входные параметры:

- индекс сервера-потребителя во внутреннем массиве компонента;
- название категории важности;
- значение категории важности.

Функция возвращает результат добавления категории в компонент:

- **«true»** - категория добавлена;
- **«false»** - категорию не удалось добавить. В таком случае следует проверить значение индекса - оно может быть больше, чем размер массива в компоненте.

AddSignal

```
bool AddSignal(int4 consumerIndex, string signalName, int4 signalMode, string signalType)
```

Добавляет в компонент информацию о сигнале, предназначенном для записи сообщений, для указанного сервера-потребителя.

Входные параметры:

- индекс сервера-потребителя во внутреннем массиве компонента;
- название сигнала;
- режим записи сообщения в сигнал:
  - **«1»** - соответствует **«DynamicEvent»** (запись сообщения с xml-конструкцией, описывающей динамическое событие);
  - **«2»** - соответствует **«Value»** (обычная запись сообщения);
- тип сообщений, записывающихся в сигнал.

Функция возвращает результат добавления информации о сигнале в компонент:

- **«true»** - информация добавлена;
- **«false»** - информацию не удалось добавить. В таком случае следует проверить значение индекса - оно может быть больше, чем размер массива в компоненте.

2.10.3. События

GenerationStarted

Начато создание текста конфигурационного файла Агент Alpha.Security.

Активируется сразу после вызова функции Generate() (стр. 145).

GenerationFinished

Создан текст конфигурационного файла Агент Alpha.Security. Результат помещен в свойство GeneratedString (стр. 145)

Активируется в результате успешного выполнения функции Generate() (стр. 145).

GenerationFailure

Не удалось создать текст конфигурационного файла Агент Alpha.Security.

Активируется в результате неуспешного выполнения функции Generate() (стр. 145).

Чтобы ознакомиться с текстом ошибки, обратитесь ко значению внутренней переменной Error этого события, либо ко значению свойства Ошибка конфигурирования (Error) (стр. 144) компонента.

ReadingStarted

Начато чтение конфигурационного файла Агент Alpha.Security.

Активируется сразу после вызова функции Read() (стр. 146).

ReadingFinished

Чтение конфигурационного файла Агент Alpha.Security завершено успешно.

Активируется в результате успешного выполнения функции Read() (стр. 146).

Чтобы ознакомиться с полученными настройками, обратитесь ко значениям свойств компонента.

ReadingFailure

Не удалось прочитать конфигурационный файл Агент Alpha.Security.

Активируется в результате неуспешного выполнения функции Read() (стр. 146).

Чтобы ознакомиться с текстом ошибки, обратитесь ко значению внутренней переменной Error этого события, либо ко значению свойства Ошибка чтения (ReadError) (стр. 145) компонента.

LdapListChanged

Изменен список LDAP-серверов во внутреннем массиве компонента.

Активируется в результате успешного выполнения одной из функций - AddLdap() или ClearLdapList() (стр. 146).

ConsumersListChanged

Изменен список серверов-потребителей сообщений аудита во внутреннем массиве компонента.

Активируется в результате успешного выполнения одной из функций - AddLogConsumer() или ClearLogConsumersList() (стр. 155).

2.11. Информация лицензирования: Получение (LicenseInfo)

Компонент предназначен для получения информации о лицензии на компоненты Альфа платформы (далее - продукты), установленные на локальной или удаленных рабочих станциях, включенных в сеть Alpha.Net.

Чтобы получить информацию о лицензии из проекта Alpha.HMI, необходимо:

1. Добавить экземпляр **`LicenseInfo`** на форму.
2. Отправить JSON-запрос серверу лицензирования Alpha.Licensing, вызвав функцию компонента RequestLicenseInfo() (или RequestRemoteLicenseInfo(), если запрос отправляется на удаленную рабочую станцию). В качестве входного параметра функции указать JSON-запрос. Шаблоны JSON-запросов приведены ниже.
3. В обработчике событий RequestLicenseInfoComplete (RequestRemoteLicenseInfoComplete) указать, что должно происходить при успешном получении JSON-ответа. Для чтения ответа можно использовать внутреннюю переменную события LicenseInfoJSON, куда записывается полученный JSON-ответ. Также ответ записывается в свойство LicenseInfo компонента. Примеры JSON-ответов приведены ниже.

   ПРИМЕЧАНИЕ

   При отправке нескольких JSON-запросов из одной команды следует учитывать разницу в чтении ответа из внутренней переменной LicenseInfoJSON и из свойства LicenseInfo.

   Значение свойства LicenseInfo перезаписывается при получении каждого нового значения, а в переменной LicenseInfoJSON полученное значение сохраняется до передачи в обработчик события. Так как процесс обработки асинхронный, то к моменту выполнения обработчика значение свойства LicenseInfo может несколько раз измениться. А значение аргумента LicenseInfoJSON будет тем же, что и в момент наступления события.

4. В обработчике событий RequestLicenseInfoFailed (RequestRemoteLicenseInfoFailed) указать, что должно происходить, если JSON-ответ от сервера получить не удалось. У события есть внутренняя переменная FailReasonCode, куда записывается номер ошибки. Чтобы получить текст ошибки по ее номеру, используйте функцию GetErrorDescriptionByCode() компонента.

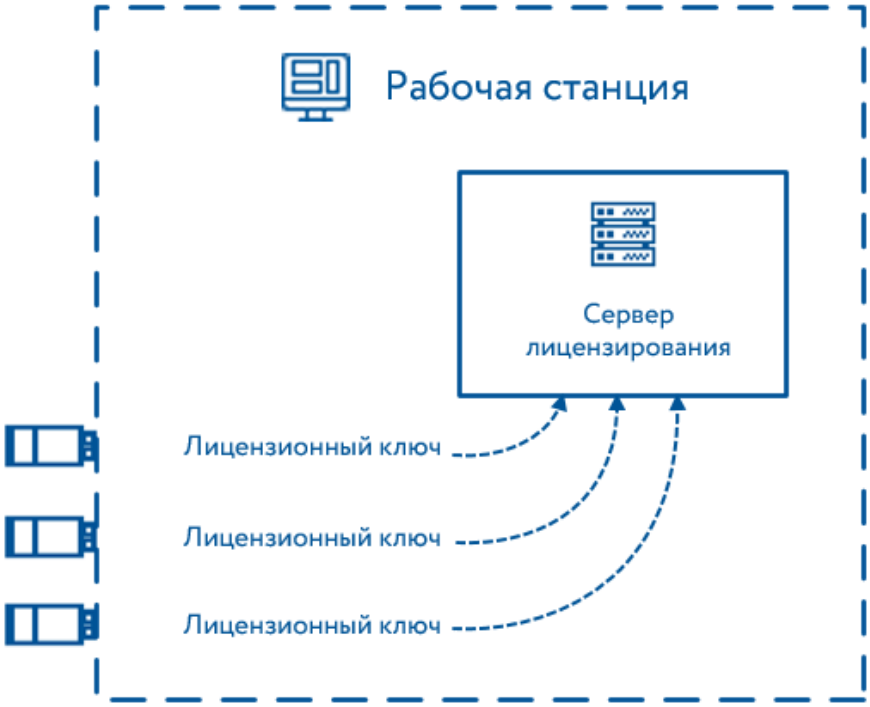
ОБРАТИТЕ ВНИМАНИЕ

Компонент может запрашивать информацию только у сервера лицензирования Alpha.Licensing версии 1.11.2 и новее.

Какую информацию предоставляет сервер лицензирования

Сервер лицензирования - это сервер, хранящий всю информацию о правах на использование продуктов Альфа платформы и/или их отдельных функций. В дальнейшем такие права будем называть лицензиями.

Лицензии предоставляются лицензионными ключами. Сервер лицензирования объединяет информацию о лицензиях, предоставляемых всеми лицензионными ключами. На каждой рабочей станции устанавливается свой сервер лицензирования.



ПРИМЕЧАНИЕ

Ключи бывают аппаратными и программными. Для лицензирования продуктов используются два вида лицензионных ключей: Sentinel и Guardant. Подробнее об этом - в документе, описывающем лицензирование Альфа платформы.

Таким образом, сервер лицензирования может предоставить:

- информацию об отдельном лицензионном ключе - его типе, о предоставляемых им лицензиях на компоненты и/или их функции;
- информацию о конкретной лицензии на продукт и/или его функцию - на определенном ключе или на устройстве в целом (со всех ключей);
- лицензионные данные - общую информацию обо всех лицензиях на продукты и/или их функции - на определенном ключе или на устройстве в целом (со всех ключей);
- полную лицензионную информацию - лицензионные данные с устройства в целом (со всех ключей).

Как построить JSON-запрос к серверу лицензирования

Чтобы построить JSON-запрос к серверу лицензирования, необходимо указать команду запроса и объект, которого касается запрос. Некоторые команды требуют указания параметров. Параметры для каждой команды описаны в подразделах с примерами запросов, приведенном ниже.

```
{
 "command": "команда",
 "object": "объект",
```

```
 "params": {}
}
```

Объект запроса - это описанная выше информация, которую предоставляет сервер лицензирования:

- Лицензионный ключ `licenseKeys`. При запросе информации о лицензионном ключе в качестве объекта запроса укажите **«licenseApi/licenseInfo/licenseKeys»**.
- Лицензия на продукт и/или его функцию `licenseFeature`. При обращении к лицензии на продукт/функцию в качестве объекта запроса укажите **«licenseApi/licenseInfo/licenseFeature»**.
- Лицензионные данные `licenseData`. При обращении к лицензионным данным в качестве объекта запроса укажите **«licenseApi/licenseInfo/licenseData»**.
- Лицензионная информация `licenseInfo`. Включает в себя лицензионные данные `licenseData` и информацию о лицензионных ключах `licenseKeys`. При обращении к полной лицензионной информации в качестве объекта запроса укажите **«licenseApi/licenseInfo/licenseInfo»**.

В запросе можно указать одну из следующих команд:

- `Get` - команда для получения информации о конкретном объекте. При использовании этой команды необходимо указывать идентификатор объекта, которого касается запрос. Команду можно отправить объектам `licenseKeys` и `licenseFeature`.
- `List` - команда для получения списка объектов конкретного типа. В ответе будут получены только значимые поля. Команду можно отправить объектам `licenseKeys`, `licenseData` и `licenseInfo`.
- `Fetch` - команда для получения списка объектов конкретного типа. В ответе будет получена вся информация об указанных объектах. Команду можно отправить объектам `licenseKeys`, `licenseData` и `licenseInfo`.

Примеры JSON-запросов и возвращаемых JSON-ответов

Ниже приведены примеры запросов к серверу лицензирования и его ответов для каждого объекта.

licenseKeys - Get: запрос информации о конкретном ключе

Чтобы получить информацию о конкретном лицензионном ключе, в параметрах запроса необходимо указать идентификатор ключа `licenseKeyId` и его класс `licenseKeyClass`.

Пример запроса:

```
{
 "command": "Get",
 "object": "licenseApi/licenseInfo/licenseKeys",
 "params":
  {
   "licenseKeyId":"0123456789",
   "licenseKeyClass":"Guardant"
  }
}
```

Пример ответа:

```
{
 "result":
 {
  "licenseKeyId":"0123456789",
  "licenseKeyClass":"Guardant",
  "licenseKeyType":"Guardant Sign",
  "licenseData":
  [
   {
    "licenseFeatureId":"11",
    "licenseFeatureDescription":"Alpha.HMI",
    "licenseFeatureCountType":"PER_DOMAIN",
    "licenseFeatureCount":"1"
   },
   {
    "licenseFeatureId":"12",
    ...
  ]
 }
}
```

Описание полей JSON-ответа

Поле	Описание
`licenseKeyId`	Идентификатор ключа.
`licenseKeyClass`	Класс ключа. Примеры значений - `HASP`, `Guardant`.
`licenseKeyType`	Тип ключа. Обычно здесь указывается, программный это ключ или аппаратный.

`licenseData`	Массив информации обо всех лицензиях на продукты и/или функции, записанных на этот ключ. Здесь:
	› `licenseFeatureId` - идентификатор лицензии на продукт и/или функцию;
	› `licenseFeatureDescription` - название продукта или его функции;
	› `licenseFeatureCountType` - тип учёта количества лицензий. Возможные значения:
	  › `PER_PROCESS` - количество лицензий на процесс;
	  › `PER_HOST` - количество лицензий на хост;
	  › `PER_DOMAIN` - количество лицензий на домен.
	› `licenseFeatureCount` - количество лицензий, предоставляемых этим ключом.

licenseKeys - List: запрос краткой информации обо всех ключах

Под краткой информацией подразумевается перечень используемых ключей, включающий идентификаторы и названия классов ключей. Чтобы получить краткую информацию обо всех лицензионных ключах, установленных на рабочую станцию, обратитесь к объекту `licenseKeys` с командой `List`.

Пример запроса:

```
{
 "command": "List",
 "object": "licenseApi/licenseInfo/licenseKeys",
}
```

Пример ответа:

```
{
 "result":
 {
  "licenseKeys":
  [
   {
    "licenseKeyId":"0123456789",
    "licenseKeyClass":"Guardant"
   },
   {
    "licenseKeyId":"0123456788",
    "licenseKeyClass":"Guardant"
   }
  ]
 }
}
```

Описание полей JSON-ответа

Поле	Описание
`licenseKeys`	Массив информации о ключах. Здесь: › `licenseKeyId` - идентификатор ключа; › `licenseKeyClass` - класс ключа. Примеры значений - `HASP`, `Guardant`.

licenseKeys - Fetch: запрос полной информации обо всех ключах

Чтобы получить полную информацию обо всех лицензионных ключах, установленных на рабочей станции, обратитесь к объекту `licenseKeys` с командой `Fetch`.

Пример запроса:

```
{
    "command": "Fetch",
    "object": "licenseApi/licenseInfo/licenseKeys"
}
```

Пример ответа:

```
{
 "result":
 {
  "licenseKeys":
  [
   {
    "licenseKeyId":"0123456789",
    "licenseKeyClass":"Guardant",
    "licenseKeyType":"Guardant Sign",
    "licenseData":
    [
     {
      "licenseFeatureId":"11",
      "licenseFeatureDescription":"Alpha.HMI",
      "licenseFeatureCountType":"PER_DOMAIN",
      "licenseFeatureCount":"1"
     },
     {
     "licenseFeatureId":"12",
     ...
    ]
   }
  ],
  [
   {
    "licenseKeyId":"0123456788",
    "licenseKeyClass":"Guardant",
    ...
   },
  ]
 }
}
```

Описание полей JSON-ответа

Поле	Описание
`licenseKeys`	Массив информации обо всех ключах, установленных на рабочей станции. Количество вложенных объектов массива (в квадратных скобках) соответствует количеству ключей.
`licenseKeyId`	Идентификатор ключа.
`licenseKeyClass`	Класс ключа. Примеры значений - `HASP`, `Guardant`.
`licenseKeyType`	Тип ключа. Обычно здесь указывается, программный это ключ или аппаратный.

Поле	Описание

`licenseData`	Массив информации обо всех лицензиях на продукты и/или функции, записанных на этот ключ. Здесь:
	› `licenseFeatureId` - идентификатор лицензии на продукт и/или функцию;
	› `licenseFeatureDescription` - название продукта или его функции;
	› `licenseFeatureCountType` - тип учёта количества лицензий. Возможные значения:
	  › `PER_PROCESS` - количество лицензий на процесс;
	  › `PER_HOST` - количество лицензий на хост;
	  › `PER_DOMAIN` - количество лицензий на домен.
	› `licenseFeatureCount` - количество лицензий, предоставляемых этим ключом.

licenseFeature - Get: запрос информации о лицензии на продукт или функцию

Чтобы получить информацию о лицензировании конкретного продукта/функции, в параметрах запроса необходимо указать источник:

- если требуется информация с определенного лицензионного ключа, в параметрах указываются идентификатор ключа `licenseKeyId`, его класс `licenseKeyClass` и идентификатор продукта/функции `licenseFeatureId`.
- если требуется полная информация со всех ключей, в параметрах указывается только идентификатор продукта/функции `licenseFeatureId`.

Пример запроса с определенного лицензионного ключа:

```
{
 "command": "Get",
 "object": "licenseApi/licenseInfo/licenseFeature",
 "params":
  {
   "licenseKeyId":"0123456789",
   "licenseKeyClass":"Guardant",
   "licenseFeatureId":"11"
  }
}
```

Пример запроса со всех ключей:

```
{
 "command": "Get",
 "object": "licenseApi/licenseInfo/licenseFeature",
 "params":
  {
   "licenseFeatureId":"11"
  }
}
```

Пример ответа:

```
{
 "result":
 {
  "licenseFeatureId":"11",
  "licenseFeatureDescription":"Alpha.HMI",
  "licenseFeatureCountType":"PER_DOMAIN",
  "licenseFeatureCount":"1"
 }
}
```

Описание полей JSON-ответа

Поле	Описание
licenseFeatureId	Идентификатор лицензии на продукт и/или функцию.
licenseFeatureDescription	Название продукта или функции.
licenseFeatureCountType	Тип учёта количества лицензий. Возможные значения: › PER_PROCESS - количество лицензий на процесс; › PER_HOST - количество лицензий на хост; › PER_DOMAIN - количество лицензий на домен.
licenseFeatureCount	Количество лицензий, предоставляемых указанным ключом или всеми ключами.

licenseData - List: запрос краткой информации обо всех лицензиях на продукты и/или функции

Под краткой информацией подразумевается перечень лицензий, включающий их идентификаторы и названия продуктов и/или функций. Чтобы получить краткую информацию обо всех лицензиях на продукты и функции, в параметрах запроса необходимо указать источник:

› если требуются лицензионные данные с определенного лицензионного ключа, в параметрах указываются идентификатор ключа licenseKeyId и его класс licenseKeyClass.

› если требуются лицензионные данные со всех ключей, параметры указывать не нужно. Обратите внимание, что результат запроса со всех ключей содержит информацию обо всех возможных лицензиях на продукты и функции - в том числе тех, которые могут быть не активны на вашем устройстве.

Пример запроса с определенного лицензионного ключа:

```
{
 "command": "List",
 "object": "licenseApi/licenseInfo/licenseData",
 "params":
  {
   "licenseKeyId":"0123456789",
   "licenseKeyClass":"Guardant"
```

```
  }
}
```

Пример запроса со всех ключей:

```
 {
  "command": "List",
  "object": "licenseApi/licenseInfo/licenseData",
 }
```

Пример ответа:

```
 {
  "result":
  {
   "licenseData":
   [
    {
     "licenseFeatureId":"11",
     "licenseFeatureDescription":"Alpha.HMI"
    },
    {
     "licenseFeatureId":"12",
     "licenseFeatureDescription":"Alpha.HMI"
    }
    ...
   ]
  }
 }
```

Описание полей JSON-ответа

Поле	Описание
`licenseFeatureId`	Идентификатор лицензии на продукт и/или функцию.
`licenseFeatureDescription`	Название продукта или функции.

licenseData - Fetch: запрос полной информации обо всех лицензиях на продукты и/или функции

Чтобы получить полную информацию обо всех лицензиях на продукты и функции, в параметрах запроса необходимо указать источник:

- если требуются лицензионные данные с определенного лицензионного ключа, в параметрах указываются идентификатор ключа `licenseKeyId` и его класс `licenseKeyClass`.
- если требуются лицензионные данные со всех ключей, параметры указывать не нужно. Обратите внимание, что результат запроса со всех ключей содержит информацию обо всех возможных лицензиях на продукты и функции - в том числе тех, которые могут быть не активны на вашем устройстве.

Пример запроса с определенного лицензионного ключа:

```
{
 "command": "Fetch",
 "object": "licenseApi/licenseInfo/licenseData",
 "params":
  {
   "licenseKeyId":"0123456789",
   "licenseKeyClass":"Guardant"
  }
}
```

Пример запроса со всех ключей:

```
{
 "command": "Fetch",
 "object": "licenseApi/licenseInfo/licenseData",
}
```

Пример ответа:

```
{
 "result":
 {
  "licenseData":
  [
   {
    "licenseFeatureId":"11",
    "licenseFeatureDescription":"Alpha.HMI",
    "licenseFeatureCountType":"PER_DOMAIN",
    "licenseFeatureCount":"1"
   },
   {
    "licenseFeatureId":"12",
    "licenseFeatureDescription":"Alpha.Server (1 000)",
    "licenseFeatureCountType":"PER_DOMAIN",
    "licenseFeatureCount":"0"
   }
   ...
  ]
 }
}
```

Описание полей JSON-ответа

Поле	Описание
licenseFeatureId	Идентификатор лицензии на продукт и/или функцию.
licenseFeatureDescription	Название продукта или функции.

Поле	Описание
`licenseFeatureCountType`	Тип учёта количества лицензий. Возможные значения: › `PER_PROCESS` - количество лицензий на процесс; › `PER_HOST` - количество лицензий на хост; › `PER_DOMAIN` - количество лицензий на домен.
`licenseFeatureCount`	Количество лицензий, предоставляемых указанным ключом или всеми ключами.

licenseInfo - List: запрос краткой лицензионной информации

Под краткой информацией подразумевается перечень ключей и предоставляемых ими лицензий. Чтобы получить краткую лицензионную информацию, обратитесь к объекту `licenseInfo` с командой `List`. Обратите внимание, что результат запроса содержит информацию обо всех возможных лицензиях на продукты и функции - в том числе тех, которые могут быть не активны на вашем устройстве.

Пример запроса:

```
{
 "command": "List",
 "object": "licenseApi/licenseInfo/licenseInfo"
}
```

Пример ответа:

```
{
 "result":
 {
  "licenseKeys":
  [
   {
    "licenseKeyId":"0123456789",
    "licenseKeyClass":"Guardant"
   },
   {
    "licenseKeyId":"0123456788",
    "licenseKeyClass":"Guardant"
   }
  ],
  "licenseData":
  [
   {
    "licenseFeatureId":"11",
    "licenseFeatureDescription":"Alpha.HMI"
   },
   {
    "licenseFeatureId":"12",
    ...
```

```
    ]
  }
}
```

Описание полей JSON-ответа

Поле	Описание
`licenseKeys`	Массив информации обо всех ключах, установленных на рабочей станции. Количество вложенных объектов массива (в квадратных скобках) соответствует количеству ключей. Здесь: › `licenseKeyId` - идентификатор ключа; › `licenseKeyClass` - класс ключа. Примеры значений - `HASP`, `Guardant`.
`licenseData`	Массив информации обо всех лицензиях на продукты и/или функции на сервере. Здесь: › `licenseFeatureId` - идентификатор лицензии на продукт и/или функцию; › `licenseFeatureDescription` - название продукта или его функции.

licenseInfo - Fetch: запрос полной лицензионной информации

Чтобы получить полную лицензионную информацию, обратитесь к объекту `licenseInfo` с командой `Fetch`. Обратите внимание, что результат запроса содержит информацию обо всех возможных лицензиях на продукты и функции - в том числе тех, которые могут быть не активны на вашем устройстве.

Пример запроса:

```
{
 "command": "Fetch",
 "object": "licenseApi/licenseInfo/licenseInfo"
}
```

Пример ответа:

```
{
 "result":
 {
  "licenseKeys":
  [
   {
    "licenseKeyId":"0123456789",
    "licenseKeyClass":"Guardant",
    "licenseKeyType":"Guardant Sign"
   },
   {
    "licenseKeyId":"0123456788",
    "licenseKeyClass":"Guardant",
    "licenseKeyType":"Guardant Sign"
   }
  ],
```

```
  "licenseData":
  [
   {
    "licenseFeatureId":"11",
    "licenseFeatureDescription":"Alpha.HMI",
    "licenseFeatureCountType":"PER_DOMAIN",
    "licenseFeatureCount":"1234567"
   },
   {
    "licenseFeatureId":"12",
    "licenseFeatureDescription":"Alpha.Server (1 000)",
    "licenseFeatureCountType":"PER_DOMAIN",
    "licenseFeatureCount":"0"
   }
   ...
  ]
 }
}
```

Описание полей JSON-ответа

Поле	Описание
`licenseKeys`	Массив информации обо всех ключах, установленных на рабочей станции. Количество вложенных объектов массива (в квадратных скобках) соответствует количеству ключей. Здесь: › `licenseKeyId` - идентификатор ключа; › `licenseKeyClass` - класс ключа. Примеры значений - `HASP`, `Guardant`; › `licenseKeyType` - тип ключа. Обычно здесь указывается, программный это ключ или аппаратный.

`licenseData`	Массив информации обо всех лицензиях на продукты и/или функции на сервере. Здесь:
	› `licenseFeatureId` - идентификатор лицензии на продукт и/или функцию;
	› `licenseFeatureDescription` - название продукта или его функции;
	› `licenseFeatureCountType` - тип учёта количества лицензий. Возможные значения:
	  › `PER_PROCESS` - количество лицензий на процесс;
	  › `PER_HOST` - количество лицензий на хост;
	  › `PER_DOMAIN` - количество лицензий на домен.
	› `licenseFeatureCount` - количество лицензий, предоставляемых этим ключом.

2.11.1. Свойства

Контекст безопасности (SecurityContext)

Ссылка на элемент типа **Контекст безопасности**, обеспечивающий взаимодействие с подсистемой безопасности Alpha.Security.

ОБРАТИТЕ ВНИМАНИЕ

Необходимо заполнить для взаимодействия с подсистемой безопасности Alpha.Security.

LicenseInfo

Результат запроса информации о лицензии. Предоставляется в формате JSON. Тип значения - string.

Только для чтения в режиме рантайма.

Примеры результатов запросов приведены на странице описания компонента (стр. 158).

2.11.2. Функции

RequestLicenseInfo

```
void RequestLicenseInfo(string jsonRequest)
```

Выполняет запрос информации о лицензии на локальной рабочей станции.

Входной параметр - JSON-запрос. Примеры запросов приведены на странице описания компонента (стр. 158).

ПРИМЕР

Вызов: LicenseInfo.RequestLicenseInfo("{\"command\":\"List\",\"object\":\"licenseApi/licenseInfo/licenseKeys\"}")

Результат:

- в случае успешного завершения операции активируется событие RequestLicenseInfoComplete (стр. 172);
- в случае неуспешного завершения операции активируется событие RequestLicenseInfoFailed (стр. 172).

ПРИМЕЧАНИЕ

Кавычки внутри JSON-запроса необходимо экранировать обратным слешем.

RequestRemoteLicenseInfo

```
void RequestRemoteLicenseInfo(string jsonRequest, string netName)
```

Выполняет запрос информации о лицензии на удаленной рабочей станции.

Входные параметры:

- JSON-запрос;
- имя рабочей станции в сети Alpha.Net.

ПРИМЕР

Вызов: LicenseInfo.RequestLicenseInfo(" {\"command\":\"List\",\"object\":\"licenseApi/licenseInfo/licenseKeys\"}", "Netname")

Результат:

- в случае успешного завершения операции активируется событие RequestRemoteLicenseInfoComplete (стр. 172);
- в случае неуспешного завершения операции активируется событие RequestRemoteLicenseInfoFailed (стр. 173).

ПРИМЕЧАНИЕ

Кавычки внутри JSON-запроса необходимо экранировать обратным слешем.

GetErrorDescriptionByCode

```
string GetErrorDescriptionByCode(uint1 FailReasonCode)
```

Возвращает текстовое описание ошибок, возникающих при запросе информации о лицензии.

Входной параметр - переменная FailReasonCode (тип значения - uint1).

2.11.3. События

RequestLicenseInfoComplete

Получена информация о лицензии на локальной рабочей станции.

Активируется в случае успешного завершения операции RequestLicenseInfo() (стр. 171).

Возвращает данные в JSON-формате во внутреннюю переменную события LicenseInfoJSON, тип значения - string.

RequestRemoteLicenseInfoComplete

Получена информация о лицензии на удаленной рабочей станции.

Активируется в случае успешного завершения операции RequestRemoteLicenseInfo() (стр. 171).

Возвращает данные в JSON-формате во внутреннюю переменную события LicenseInfoJSON, тип значения - string.

RequestLicenseInfoFailed

Не удалось получить информацию о лицензии на локальной рабочей станции.

Активируется в случае неуспешного завершения операции RequestLicenseInfo() (стр. 171).

Возвращает номер ошибки во внутреннюю переменную события FailReasonCode, тип значения - uint1.

RequestRemoteLicenseInfoFailed

Не удалось получить информацию о лицензии на удаленной рабочей станции.

Активируется в случае неуспешного завершения операции RequestRemoteLicenseInfo() (стр. 171).

Возвращает номер ошибки во внутреннюю переменную события FailReasonCode, тип значения - uint1.

История изменений

2.0

Важно. Текущая версия предназначена для использования с Alpha.HMI 2.0.
Улучшение
Функция GetMembersList() компонента 2.8. Настройка безопасности: Пользователь (SecurityManagerUser) (стр. 120) теперь предоставляет больше данных. Помимо uid и логина (или названия) участника группы теперь предоставляется информация о том, является ли участник пользователем или дочерней группой.
Исправление
Устранена причина, по которой значения логических прав не сбрасывались после выхода пользователя из подсистемы.

2.0.1

Улучшение

Функция RequestUserList() компонента 2.6. Настройка безопасности: Менеджер (SecurityManager) (стр. 96), помимо прочих данных об учетных записях, теперь предоставляет их уникальные идентификаторы (uid).

Исправления

Доработаны функции и события компонента 2.1. Контекст безопасности (SecurityContext) (стр. 50):

- При обращении функции RemoteGetLoggedUsersList() к недоступной рабочей станции не активировалось событие RemoteGetLoggedUsersFailed.
- События ConnectedChanged и CurrentUserChanged активировались дважды при запуске проекта Alpha.HMI в рантайм, теперь - только один раз.

2.0.2

Функция GetTokensList() компонента **Настройка безопасности: Приложение** теперь предоставляет больше данных о правах приложений.

2.0.3

Обратите внимание: не рекомендуется использовать предыдущую версию 2.0.2 из-за иногда возникающих ошибок запуска проектов Alpha.HMI, использующих компоненты безопасности.

Новая возможность

Разработан компонент **Информация лицензирования: Получение (LicenseInfo)**. Компонент предназначен для получения информации о лицензии на продукты Альфа платформы, установленные на локальной или удаленных рабочих станциях, включенных в сеть Alpha.Net.

Исправления

- Устранена причина, по которой недавно удаленная учетная запись появлялась в списке пользователей, полученном в результате вызова функции RequestUsersList() компонента **Настройка безопасности: Менеджер (SecurityManager)**.

- Метод проверки соответствия пароля требованиям ValidatePassword() компонента **Настройка безопасности: Пользователь (SecurityManagerUser)** теперь корректно работает и для кириллических паролей.

2.0.4

Исправления

- Исправлена ошибка, из-за которой был невозможен вход под учетной записью, которой требовалась смена пароля, во время другой активной пользовательской сессии.
- Устранена причина активации двух событий компонента **Настройка безопасности: Менеджер** при вызове функции ExportConfiguration() - успешное и неуспешное сохранение резервной копии конфигурации - если сохранить копию пытался пользователь без прав на создание файлов в указанной папке.
- Теперь при сохранении учетной записи пользователя с помощью функции Save() компонента **Настройка безопасности: Пользователь** выполняется проверка, не указан ли пустой пароль.
- Исправлена ошибка, из-за которой у пользователя не учитывались права, назначенные его группе из приложения, если этой группе также была назначена роль из этого приложения.
- Устранена причина, по которой функция GetApplicationsList() компонента **Настройка безопасности: Группа** не возвращала список прав группы, унаследованных от родительской группы.
- Исправлена ошибка, из-за которой у компонента **Настройка безопасности: Контроль целостности** активировались события об успешном завершении операции при обращении к несуществующему узлу.
- События ConnectedChanged и CurrentUserChanged компонента **Контекст безопасности** снова активируются дважды при запуске проекта Alpha.HMI в рантайм. Два события - это более правильное поведение компонента. В первый раз событие активируется при начальной инициализации проекта. Второй раз - когда агент передает в проект информацию о текущем пользователе.
- Исправлены другие некритичные ошибки.

2.0.5

Исправление

Устранена причина, по которой функция GroupDisplayName() компонента **Контекст безопасности** не возвращала значений. Ошибка возникала при использовании предыдущей версии - 2.0.4.

2.0.7

Исправления

- Устранена причина, по которой свойство GuestMode компонента **Контекст безопасности** предоставляло некорректное значение при первом запуске проекта в рантайм.
- Функция ValidatePassword() компонента **Настройка безопасности: Пользователь** теперь возвращает корректные значения, если проверяемый пароль пустой или содержит пробелы.
- Исправлена ошибка, из-за которой права, назначенные роли, не удалялись при использовании функции RoleDeleteRight() компонента **Настройка безопасности: Приложение**.

2.0.8

Исправления

- Устранена причина, по которой при неуспешном создании эталона активировалось событие CreateFinished компонента 2.3. Настройка безопасности: Контроль целостности (SecurityIntegrityControl) (стр. 68).
- Исправлена ошибка, возникавшая при попытке указать ссылку на экземпляр типа 2.1. Контекст безопасности (SecurityContext) (стр. 50) из свойства какого-либо компонента.

2.0.9

Новые возможности

Добавлена возможность подключения к произвольному серверу безопасности. Для этого компоненту **Настройка безопасности: Менеджер** добавлены новые:

- свойства CurrentLdapHost, CurrentLdapPort - для получения информации о текущем LDAP-сервере;
- функции SetConfigureServer(), ResetConfigureServer() - для подключения и отключения от LDAP-сервера;
- события LDAPServerChanged, LDAPServerChangeFailure - для получения информации о подключении к LDAP-серверу.

Улучшение

При создании учетной записи с помощью компонента **Настройка безопасности: Пользователь** теперь выполняется проверка уникальности указанных логинов и отображаемых имен.

Исправления

- Устранена причина, по которой функция RequestUsersList() компонента **Настройка безопасности: Менеджер** не предоставляла обновленный список пользователей после импорта учетных записей из домена.
- Исправлена некорректная активация событий DeleteUserComplete и DeleteUserFailed компонента **Настройка безопасности: Менеджер** при попытке удаления учетной записи с правом **Запрещено удалять пользователя если есть активная сессия (NonDeletebleWhileActive)**.
- Исправлен текст ошибки, возникающей при попытке повторного входа пользователя с правом **Только одна пользовательская сессия (ActivityLimitedToOneWorkstation)**. Ранее текст ошибки был неинформативен.
- Устранена ошибка, возникавшая при попытке создать приложение или право с длинным названием: допустимое количество символов увеличено до 100.

2.0.10

Улучшение

Компоненту **Настройки безопасности: Контроль целостности** добавлены новые функции и события:

- Функция SubscribeEtalonDiffConfig() предназначена для подписки на событие изменения файла конфигурации контроля целостности, функция UnsubscribeEtalonDiffConfig() - для отмены подписки. Можно не указывать входной параметр - тогда будет меняться состояние подписки на изменения локального файла; либо указать имя удаленной рабочей станции для подписки на изменения удаленного файла.
- События EtalonDiffConfig и RemoteEtalonDiffConfig, активирующиеся при получении уведомления об изменении файла конфигурации контроля целостности на локальной или удаленной станции соответственно.

- События EtalonDiffConfigFailed и RemoteEtalonDiffConfigFailed, активирующиеся при возникновении проблем при получении уведомления об изменении файла конфигурации контроля целостности на локальной или удаленной станции соответственно.

Исправления

- Устранены ошибки, связанные с выполнением функции SetConfigureServer() компонента **Настройка безопасности: Менеджер**:
  - Увеличен таймаут запроса данных с удаленного LDAP-сервера. Ранее запрос часто завершался с ошибкой "Ответ не получен за отведенное время".
  - Устранена причина, по которой при подключении к удаленному LDAP-серверу сохранение и восстановление конфигурации из резервной копии ошибочно выполнялось для локального сервера.
- Исправлен неинформативный текст ошибки, возникающей при попытке редактирования конфигурации от имени пользователя, имеющего права на просмотр, но не редактирование конфигурации.
- Устранена причина аварийного завершения работы Alpha.HMI при указании слишком длинного пароля в качестве значения входного параметра функции AsyncLogin() компонента **Контекст безопасности**.

2.0.12

Исправления

Исправлена некорректная работа следующих компонентов:

- **Настройка безопасности: Контроль целостности**, событие EtalonDiffConfig.
- **Настройка безопасности: Приложения**, функция SaveFalied() - при попытке изменить значение права роли.
- **Настройка безопасности: Пользователь**, функция Save() - при попытке изменить пароль пользователя после ужесточения требований к паролю.

Включены изменения непубличной версии 2.0.11

Улучшения

- Компоненту **Настройка безопасности: Контроль целостности** добавлены новые функции и события:
  - функции JsonRequest() и JsonRemoteRequest(), предназначенные для получения и изменения текущей конфигурации контроля целостности в виде JSON-строки на локальной или удаленной рабочей станции;
  - события JsonRequestComplete и JsonRemoteRequestComplete, активирующиеся при получении текущей конфигурации контроля целостности в переменную response или в случае возникновения проблем при ее получении;
  - функции SubscribeAutoControlResults() и UnsubscribeAutoControlResults(), предназначенные для подписки и отмены подписки на результаты автоматических проверок целостности;
  - события AutoCheckComplete и AutoCheckFailed, активирующиеся при получении результата автоматической проверки целостности в переменную autoResult или в случае возникновения проблем при его получении.

  Также у компонента удалены события RemoteEtalonDiffConfig и RemoteEtalonDiffConfigFailed, добавленные в Alpha.HMI.Security 2.0.10. Теперь уведомление об изменении файла конфигурации контроля целостности на локальной или удаленной станции активирует одно и то же событие - EtalonDiffConfig, а ошибка получения уведомления - событие EtalonDiffConfigFailed.

- Компоненту **Контекст безопасности** добавлено свойство **Дополнительная проверка криптостойкости пароля (ExtendPasswordStrengthCheck)**, запускающее дополнительную проверку криптостойкости пароля.

Исправления

- Устранена причина, по которой при подключении к каталогу на удаленном LDAP-сервере пользователю, не имеющему доступа к просмотру и редактированию конфигурацию каталога, такой доступ ошибочно предоставлялся.
- Попытка сохранить значение строкового права с пустыми строками приводила к исчезновению этого права.
- Наличие экземпляра типа **Контекст безопасности** в проекте приводило к аварийному завершению работы Alpha.HMI при перезапуске службы Alpha.Net.

2.0.13

Для корректной работы этой версии используйте Alpha.Security 1.7.26.

Новая возможность

Компоненту 2.6. Настройка безопасности: Менеджер (SecurityManager) (стр. 96) добавлены:

- функция **GetExternalUserList()**, запрашивающая список учетных записей из внешнего источника аутентификации, указанного в настройках агента безопасности;
- событие **GetExternalUserListFinished**, активирующееся в случае успешного получения списка учетных записей, запрошенных функцией **GetExternalUserList()**.

Исправления

- Устранена причина, по которой компоненты расширения не получали актуальных значений прав для текущего пользователя из удаленного LDAP-сервера.
- Импорт конфигурации из резервной копии на удаленном LDAP-сервере теперь возможен только при предоставлении учетных данных администратора этого сервера.
- Эффективные значения прав системного приложения **Alpha.Security** теперь высчитываются корректно.

2.0.15

Исправление

[Контроль целостности] Устранена причина аварийного завершения работы Alpha.HMI при попытке создания эталона с использованием компонентов расширения.

Изменения документации

Редакция 1

Обновлен пример возвращаемого значения:

- для функции **GetMembersList()** компонента 2.9. Настройка безопасности: Группа (SecurityManagerGroup) (стр. 131).

- для функции **RequestUsersList()** компонента 2.6. Настройка безопасности: Менеджер (SecurityManager) (стр. 96). Пример приведен в описании события RequestUsersListComplete (стр. 104), активирующегося в случае успешного выполнения функции.

Редакция 2

В разделе 1.6. Контроль целостности файлов и папок (стр. 35) руководства пользователя:

- актуализировано описание настройки контроля целостности со стороны Alpha.Security;
- добавлен новый подраздел, в котором приведена расшифровка сообщений о проверке целостности файлов (стр. 39).

В справочном руководстве актуализировано описание свойства IC_List_JSON (стр. 79) компонента **Настройка безопасности: Контроль целостности**.

Редакция 3

Внутренние изменения. Содержимое документа не изменилось.

Редакция 4

В справочном руководстве описан новый компонент - 2.11. Информация лицензирования: Получение (LicenseInfo) (стр. 158).

Редакция 5

В руководстве пользователя:

- В раздел 1.2. Подготовка к работе (стр. 6) добавлены команды установки и удаления веб-версии расширения на ОС Linux.
- Обновлен проект-пример, прикладываемый к разделу 1.3. Управление доступом в проектах (стр. 8).
- Обновлены скриншоты.

В справочном руководстве:

- Описана ранее не описанная функция компонента **Настройка безопасности: Пользователь** - ValidatePassword() (стр. 123). Здесь же актуализировано описание функции **SetPassword()**.
- Обновлен пример значения, возвращаемого функцией GetTokensList() (стр. 112) компонента **Настройка безопасности: Приложение**.

Редакция 6

Внутренние изменения. Содержимое документа не изменилось.

Редакция 7

Добавлен новый раздел 1.8. Логирование сообщений безопасности (стр. 46). Раздел описывает настройку получения сообщений о действиях пользователя в проекте Alpha.HMI в журнал сообщений с помощью компонента **Контекст безопасности**.

Редакция 8

В руководстве пользователя обновлен раздел, описывающий, как создать 1.5. Собственный конфигуратор подсистемы безопасности (стр. 22): добавлен подраздел, описывающий настройку подключения к произвольному LDAP-серверу. Также обновлен прикладываемый к разделу проект-пример.

Актуализировано справочное руководство.

Редакция 9

В руководстве пользователя обновлен раздел 1.6. Контроль целостности файлов и папок (стр. 35): переработана структура раздела, добавлен пример использования компонента 2.3. Настройка безопасности: Контроль целостности (SecurityIntegrityControl) (стр. 68) для получения текущей конфигурации контроля целостности, обновлен проект-пример, прикладываемый к разделу.

Также актуализировано справочное руководство:

- Для компонента 2.1. Контекст безопасности (SecurityContext) (стр. 50) описано новое свойство **Дополнительная проверка криптостойкости пароля (ExtendPasswordStrengthCheck)**.
- В разделе, описывающем компонент 2.3. Настройка безопасности: Контроль целостности (SecurityIntegrityControl) (стр. 68), описано, как построить запрос к агенту безопасности для получения и изменения текущей конфигурации контроля целостности с помощью экземпляра компонента (для Alpha.Security версии 1.7.24 и новее); также описаны новые функции и события.

Редакция 10

В справочном руководстве актуализировано описание компонента 2.6. Настройка безопасности: Менеджер (SecurityManager) (стр. 96), описаны его новые функция GetExternalUserList() и событие GetExternalUserListFinished.

Редакция 11

В справочном руководстве:

- Описаны новая функция **LogAudit2()** элемента **Контекст безопасности** и новое событие **LdapUserChanged** элемента **Настройка безопасности: Менеджер**.
- В описаниях свойств, функций и событий, обрабатывающих строковые права, упомянута возможность загрузки нескольких значений.

1.2

1.2.3

Улучшение

Функция **RequestUserList()** компонента 2.6. Настройка безопасности: Менеджер (SecurityManager) (стр. 96) теперь предоставляет больше данных. Раньше список, возвращаемый функцией, содержал только uid и логин каждой учетной записи подсистемы безопасности. Теперь список содержит больше данных: фамилии, должности, номера телефонов и пр.

Исправление
Компонент 2.1. Контекст безопасности (SecurityContext) (стр. 50) больше не посылает в журнал приложений сообщения о внутренней работе. К таким относились сообщения вида "Деструктуризация булевского токена ...", "Удаление указателя на прокси контекст у токена ..." и т.п.

Изменения документации

Редакция 1

В руководстве пользователя более подробно описан процесс подготовки конфигурационных файлов Alpha.Security для настройки контроля целостности (стр. 35).

В справочном руководстве:

- Добавлено описание нового события LastActionError компонента **Настройка безопасности: Менеджер** (стр. 108). Событие активируется при возникновении ошибки в процессе выполнения последнего действия.
- Для компонента **Контекст безопасности** описаны новые:
  - функции RemoteGetLoggedUsersList, GetRemoteLoggedUserCount и GetRemoteLoggedUserByIndex (стр. 61);
  - события RemoteGetLoggedUsersFinished и RemoteGetLoggedUsersFailed (стр. 64).

  С их помощью можно узнать информацию о текущих пользователях удаленных рабочих станций.
- Добавлено описание свойства **Disabled (Пользователь заблокирован)** для компонентов Настройка безопасности: Пользователь (стр. 121) и Настройка безопасности: Группа (стр. 132). Свойство хранит информацию о том, заблокирован ли пользователь/группа.
- В описание функций всех компонентов для удобства использования добавлены сигнатуры.

Во всем документе скриншоты, содержащие код обработчиков событий, заменены на текстовые блоки кода.

Редакция 2

Описан ранее не описанный компонент **Мастер конфигурирования Security**:

- В руководство пользователя добавлен новый раздел 1.7. Конфигурирование агента безопасности (стр. 40), описывающий пример использования компонента в проектах Alpha.HMI.
- В справочное руководство добавлено описание свойств и методов компонента.

Редакция 3

В руководстве пользователя:

- В раздел 1.2. Подготовка к работе (стр. 6) добавлена информация об установке и удалении Alpha.HMI.Security на ОС Linux.
- Обновлен проект-пример в разделе 1.5. Собственный конфигуратор подсистемы безопасности (стр. 22). Для всех форм проекта создан общий **Контекст безопасности**. Он помещен в глобальные объекты.

В справочном руководстве:

- Обновлен пример значения, возвращаемого функцией **RequestUserList()** компонента 2.6. Настройка безопасности: Менеджер (SecurityManager) (стр. 96). Пример приведен в описании события RequestAppListComplete (стр. 103), активирующегося в результате успешного выполнения функции.

- Добавлена информация об особенности события CurrentUserChanged (стр. 63) - оно активируется дважды при запуске проекта Alpha.HMI в рантайм.

Также исправлена история изменений для второй редакции документа, соответствующей Alpha.HMI.Security версии 1.2. Ранее было указано, что содержимое документа не изменилось.

1.1

1.1.1

Реализована возможность создания и восстановления резервных копий конфигурации Alpha.Security (стр. 32).

Изменения документации

Редакция 2

Руководство пользователя

- Описана возможность создания и восстановления резервных копий конфигурации Alpha.Security (стр. 32). Возможность работы с резервными копиями также реализована в проекте-примере, прилагаемом к разделу.
- Стилизована иллюстрация схемы взаимодействия Alpha.HMI и Alpha.Security с помощью Alpha.HMI.Security (стр. 5).

Справочное руководство

- Описаны новые функции и события компонента **Настройка безопасности: Менеджер**, используемые для создания и восстановления резервных копий конфигурации Alpha.Security.
- Исправлено неверно указанное имя входного параметра функции GetErrorDescriptionByCode() компонентов **Настройка безопасности: Приложение**, **Настройка безопасности: Группа**, **Настройка безопасности: Пользователь**: параметр называется не FailReason, а FailReasonCode.
- Исправлено описание функции LogAudit(): входные параметры могут принимать значения **«от 0 до 3»**, а не **«от 1 до 4»**, как было указано ранее.
- Дополнены примеры возвращаемых значений некоторых функций.

1.0

Изменения документации

Редакция 2

- Реализована возможность скачивания файлов проектов-примеров из веб-версии документа.
- Обновлена информация о количестве юнитов, появляющихся при установке расширения. Вместо двух юнитов - **Security** и **Security Configuration** - появляется один юнит **Security**.

- Добавлено описание функции **Login()** (стр. 46) компонента **Контекст безопасности** (**SecurityContext**).
- Исправлен неверно указанный входной параметр функций **ForceStopUserSession()** (стр. 61) и **ResetUserFailedLoginCounter()** (стр. 61) компонента **Контекст безопасности** (**SecurityContext**). Входным параметром обеих функций является логин пользователя, а не uid, как было указано ранее.

Программный комплекс Альфа платформа

Alpha.HMI.SecurityConfigurator 3.0

Руководство пользователя

Редакция	Соответствует версии ПО
1	3.0.0

© АО «Атомик Софт», 2015-2026. Все права защищены.

Авторские права на данный документ принадлежат АО «Атомик Софт». Копирование, перепечатка и публикация любой части или всего документа не допускается без письменного разрешения правообладателя.

Содержание

1. Руководство пользователя 5
1.1. О продукте 5
1.2. Подготовка к работе 6
1.3. Запуск самостоятельного приложения 7
1.4. Встраивание в проект 8
1.5. Вход с учетными данными 10
1.6. Подключение к произвольному каталогу на текущем или удаленном LDAP-сервере 11
1.7. Редактирование конфигурации 13
1.7.1. Создание и редактирование групп пользователей 13
1.7.2. Создание приложений и прав 14
1.7.3. Назначение прав 16
1.7.4. Создание ролей 17
1.7.5. Создание и редактирование учетных записей пользователей 18
1.7.6. Вычисление эффективных значений прав 22
1.7.7. Безопасность в компонентах Альфа платформы 23
1.7.8. Резервное копирование конфигурации 25
1.8. Настройки безопасности для разграничения возможностей пользователей 26
1.9. Приложения 28
1.9.1. Права системного приложения Alpha.Security 28

2. Справочное руководство 33
2.1. SecurityConfigurator 33
2.1.1. Свойства 33
2.2. Настройки 34
2.2.1. Свойства 34
2.3. Разрешения 35
2.3.1. Свойства 36
2.4. SecurityConfigurator_App 36
2.5. SecurityConfigurator_Page 36
2.5.1. Свойства 37
2.6. SecurityConfigurator_Form 37
2.6.1. Свойства 38
2.7. SecurityConfigurator_Tests 38

История изменений 39
3.0 39
Изменения документации 39
Редакция 1 39
2.5 40
Изменения документации 40
Редакция 1 40
2.4 40
Изменения документации 41
Редакция 1 41
2.3 41
2.3.1 42
2.3.2 42
2.3.3 42
Изменения документации 43
Редакция 1 43
Редакция 2 43
Редакция 3 43
2.2 43

2.2.1 44
2.2.2 44
Изменения документации 44
Редакция 1 44
2.1 45
2.1.1 45
2.1.2 46
Изменения документации 46
Редакция 2 46
Редакция 3 46
2.0 47
Изменения документации 47
Редакция 1 47
1.3 47
Изменения документации 47
Редакция 1 47
Редакция 2 47
1.2 47
1.2.1 48
Изменения документации 48
Редакция 1 48
Редакция 2 48
Редакция 3 48
1.1 49
1.1.1 49
Изменения документации 49
Редакция 1 49
Редакция 2 49

1. Руководство пользователя

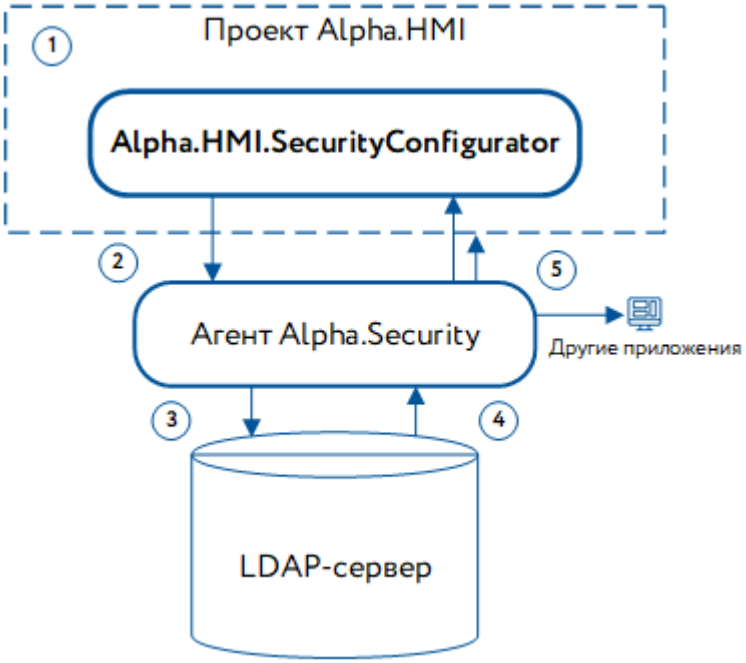
1.1. О продукте

Alpha.HMI.SecurityConfigurator - приложение, предназначенное для конфигурирования подсистемы безопасности Alpha.Security.

Под конфигурированием подсистемы безопасности подразумевается:

- создание учетных записей пользователей для предоставления им доступа к возможностям проекта;
- объединение пользователей в группы для предоставления им одинаковых возможностей;
- создание прав доступа к возможностям проекта и группировка прав в приложения;
- создание ролей и назначение их пользователям или группам;
- назначение прав пользователям, группам и/или ролям.

Используйте Alpha.HMI.SecurityConfigurator как самостоятельное приложение (стр. 7), или встраивайте его в проекты автоматизации (стр. 8), разработанные в Alpha.HMI.
Роль Alpha.HMI.SecurityConfigurator в схеме взаимодействия с Alpha.Security и другими приложениями можно описать следующей иллюстрацией:



1. Alpha.HMI.SecurityConfigurator встраивается в проект автоматизации, реализованный в среде разработки Alpha.HMI, или вызывается как самостоятельное приложение.
2. С помощью Alpha.HMI.SecurityConfigurator меняется конфигурация подсистемы безопасности Alpha.Security. Новая конфигурация передается Агент Alpha.Security.
3. Агент Alpha.Security записывает на LDAP-сервер конфигурацию, где она хранится в виде каталогов LDAP.
4. LDAP-сервер по запросу предоставляет информацию о пользователях, группах, ролях и их возможностях Агент Alpha.Security.
5. Агент Alpha.Security предоставляет информацию всем приложениям, запрашивающим ее.

Подробнее с принципами организации и работы подсистемы безопасности Alpha.Security можно ознакомиться в руководстве администратора Alpha.Security.

1.2. Подготовка к работе

Требования к окружению

Для работы Alpha.HMI.SecurityConfigurator должны быть установлены:

- Alpha.HMI - среда разработки проектов автоматизации;
- Alpha.Security - подсистема безопасности, которую можно конфигурировать в соответствии с нуждами проекта;
- Alpha.Domain - компонент, обеспечивающий взаимодействие между Alpha.HMI и Alpha.Security;
- Alpha.HMI.Security - компонент, обеспечивающее взаимодействие приложения с Alpha.Security;
- Alpha.HMI.Tables - компонент, обеспечивающий отображение компонентов приложения в проекте в режиме исполнения.

Если приложение используется в веб-версии проекта автоматизации, то установите дополнительно:

- Alpha.HMI.WebViewer - для просмотра проектов Alpha.HMI в веб-интерфейсе;
- Alpha.HMI.Security.WebViewer - для работы компонентов Alpha.HMI.Security в веб-интерфейсе;
- Alpha.HMI.Tables.WebViewer - для работы компонентов Alpha.HMI.Tables в веб-интерфейсе.

Если необходимо подключить Alpha.HMI.SecurityConfigurator к проекту Alpha.HMI, установите также Alpha.HMI.CommonLib.

Установка, удаление или восстановление

ОС Windows

Для установки, удаления или восстановления Alpha.HMI.SecurityConfigurator запустите установочный файл `alpha.hmi.securityconfigurator-<lng>-<version>.<arch>.msi` и следуйте инструкциям мастера.

> ПРИМЕЧАНИЕ
> В названии файла `<lng>` - это язык компонента, `<version>` - номер версии компонента, а `<arch>` - целевая процессорная архитектура.

ОС Linux

> ОБРАТИТЕ ВНИМАНИЕ
> Команды на установку и удаление нужно запускать с правами суперпользователя.

Для установки Alpha.HMI.SecurityConfigurator вызовите пакет с командой на установку в зависимости от используемого пакетного менеджера:

- rpm-пакет:

```
sudo rpm -i alpha.hmi.securityconfigurator-<lng>-<version>.rpm
```

- deb-пакет:

```
sudo dpkg -i alpha.hmi.securityconfigurator-<lng>-<version>.deb
```

ПРИМЕЧАНИЕ

В названии пакета `<lng>` - это язык компонента, а `<version>` - номер версии компонента.

Для удаления Alpha.HMI.SecurityConfigurator вызовите команду на удаление приложения в зависимости от используемого пакетного менеджера:

- rpm-пакет:

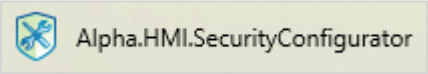
```
sudo rpm -e alpha.hmi.securityconfigurator
```

- deb-пакет:

```
sudo dpkg -r alpha.hmi.securityconfigurator
```

1.3. Запуск самостоятельного приложения

Чтобы запустить Alpha.HMI.SecurityConfigurator как самостоятельное приложение, нажмите иконку приложения в меню `Пуск`.



Откроется окно Alpha.HMI.SecurityConfigurator, настройки которого можно изменить:

- изменить тему оформления;
- добавить кнопку вызова экранной клавиатуры в строку состояния и диалоговые окна;
- ограничить доступ пользователей к файловой системе в диалоговых окнах сохранения и открытия файлов. Подробнее об этом - в разделе 1.8. Настройки безопасности для разграничения возможностей пользователей (стр. 26).

Чтобы сразу приступить к просмотру и редактированию конфигурации, необходимо войти с учетными данными администратора (стр. 10).

Чтобы сначала изменить настройки окна приложения Alpha.HMI.SecurityConfigurator, перейдите к файлу `app_settings.json`, расположенному в:

- `C:\ProgramData\Automiq\HMI.SecurityConfigurator` для ОС Windows;
- `/home/<user>/Automiq/HMI.SecurityConfigurator` для ОС Linux.

ОБРАТИТЕ ВНИМАНИЕ

Перед редактированием файла закройте приложение.

Изменение темы оформления

Чтобы изменить тему оформления приложения, в конфигурационном файле `app_settings.json` измените значения свойств объекта `Theme`. Свойства объекта соответствуют свойствам типа **Тема оформления**, предоставляемого расширением Alpha.HMI.CommonLib. Подробнее со свойствами можно ознакомиться в документации для Alpha.HMI.CommonLib.

После изменения значений свойств сохраните файл и перезапустите приложение. Его внешний вид будет изменен.

Добавление кнопки вызова экранной клавиатуры

Чтобы добавить в строку состояния кнопку вызова экранной клавиатуры:

1. В конфигурационном файле `app_settings.json` измените значения свойств объекта `ScreenKeyboard` следующим образом:
   - Включите вызов экранной клавиатуры, изменив значение свойства `Enabled` с **«false»** на **«true»**.
   - Укажите полный путь к программе экранной клавиатуры в качестве значения свойства `ProgramPath`. Новое значение можно не указывать, тогда по умолчанию будет использоваться:
     - в Windows - `alpha.hmi.keyboard.exe` (поставляется вместе с дистрибутивом Alpha.HMI).
     - в Linux - `/usr/bin/fly-vkbd`.
   - Укажите аргументы запуска программы экранной клавиатуры в качестве значения свойства `Arguments` (зависят от используемой программы).
2. Сохраните файл и перезапустите приложение.

В строке состояния и во всех диалоговых окнах, где требуется ввод значений, появится кнопка вызова указанной экранной клавиатуры.

Настройки сессии приложения

Приложение также сохраняет файл `app_session.json` - он хранит данные о размере и положении окна приложения во время последней сессии приложения.

Файл сохраняется в:

- `C:\Users\<user>\Automiq\HMI.SecurityConfigurator` для ОС Windows;
- `/home/<user>/Automiq/HMI.SecurityConfigurator` для ОС Linux.

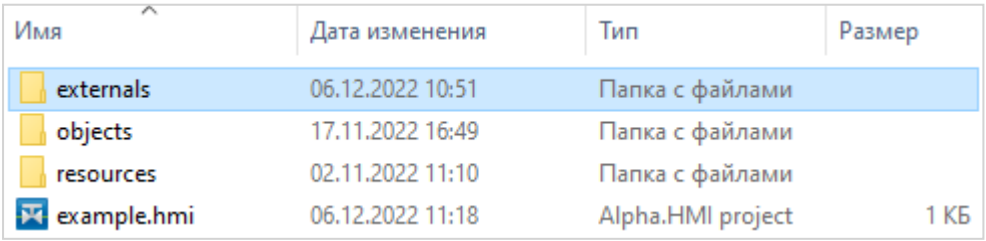
Файл используется для работы приложения, менять его содержимое не рекомендуется.

1.4. Встраивание в проект

Чтобы встроить Alpha.HMI.SecurityConfigurator в проект автоматизации, разработанный в Alpha.HMI, подключите проект расширения в свой проект в виде внешнего модуля.

> ОБРАТИТЕ ВНИМАНИЕ
> Для встраивания Alpha.HMI.SecurityConfigurator в проект Alpha.HMI понадобится библиотека Alpha.HMI.CommonLib.

1. Чтобы подключать проекты в виде внешних модулей, создайте в папке своего проекта папку с именем `externals`.

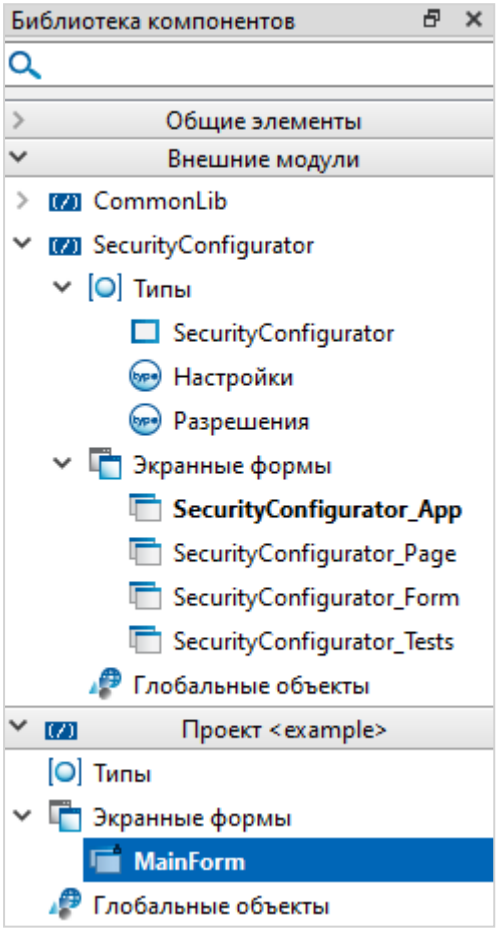


Имя	Дата изменения	Тип	Размер
externals	06.12.2022 10:51	Папка с файлами	
objects	17.11.2022 16:49	Папка с файлами	
resources	02.11.2022 11:10	Папка с файлами	
example.hmi	06.12.2022 11:18	Alpha.HMI project	1 КБ

2. Перейдите к папке с расширениями Alpha.HMI:
   - `C:\Program Files\Automiq\Alpha.HMI.Extensions` или `C:\Program Files (x86)\Automiq\Alpha.HMI.Extensions` (ОС Windows);
   - `/opt/Automiq/Alpha.HMI.Extensions` (ОС Linux).

Отсюда скопируйте папку `SecurityConfigurator` и папку библиотеки `CommonLib` в созданную ранее `externals`.

3. Запустите свой проект в Дизайнер Alpha.HMI. В меню **Проект** на панели инструментов выберите **Обновить ссылки на внешние юниты**. В результате в библиотеке компонентов появятся юниты внешних модулей - **SecurityConfigurator** и **CommonLib**.



Внешний модуль **SecurityConfigurator** предоставляет типы и экранные формы.

- Чтобы добавить экземпляр типа в проект, просто перетяните его на рабочую область.
- Чтобы использовать экранную форму в проекте, вызовите ее в обработчике какого-либо события или перетяните на рабочую область.

С назначением типов и экранных форм можно ознакомиться в Справочном руководстве.

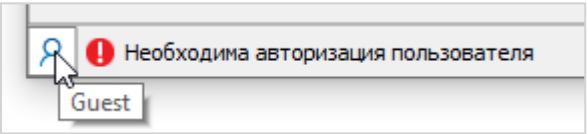
Обновить Alpha.HMI.SecurityConfigurator в проекте Alpha.HMI

Если Alpha.HMI.SecurityConfigurator уже встроен в проект в виде внешнего модуля, то при установке новой версии необходимо обновить файлы модуля в папке `externals` проекта. Для этого:

1. Установите нужную версию Alpha.HMI.SecurityConfigurator.
2. Замените существующую папку `SecurityConfigurator` на папку `SecurityConfigurator`, расположенную в папке установки расширения:
   - `C:\Program Files\Automiq\Alpha.HMI.Extensions` или `C:\Program Files (x86)\Automiq\Alpha.HMI.Extensions` (ОС Windows);
   - `/opt/Automiq/Alpha.HMI.Extensions` (ОС Linux).
3. Если устанавливали новую версию Alpha.HMI.CommonLib, не забудьте аналогично обновить файлы модуля в папке `externals` проекта.
4. Запустите свой проект в Дизайнер Alpha.HMI. В меню **Проект** на панели инструментов выберите **Обновить ссылки на внешние юниты**. В результате внешние модули будут обновлены. Экранные формы и типы, предоставляемые расширением и уже добавленные в проект ранее, обновятся автоматически.

1.5. Вход с учетными данными

Чтобы приступить к работе, необходимо авторизоваться. Для этого нажмите на иконку в левом нижнем углу.



Откроется окно авторизации пользователя, где нужно указать логин и пароль учетной записи администратора каталога LDAP, к которому выполнено подключение. Введите учетные данные администратора каталога и нажмите **Войти в систему**. Если введенные данные верны, загрузится окно просмотра учетных записей, будут доступны просмотр и редактирование конфигурации. В противном случае доступ будет отклонен, и в окне конфигуратора появится сообщение об этом.

Если вам необходимо подключиться к другому каталогу, перейдите к разделу 1.6. Подключение к произвольному каталогу на текущем или удаленном LDAP-сервере (стр. 11).

ОБРАТИТЕ ВНИМАНИЕ

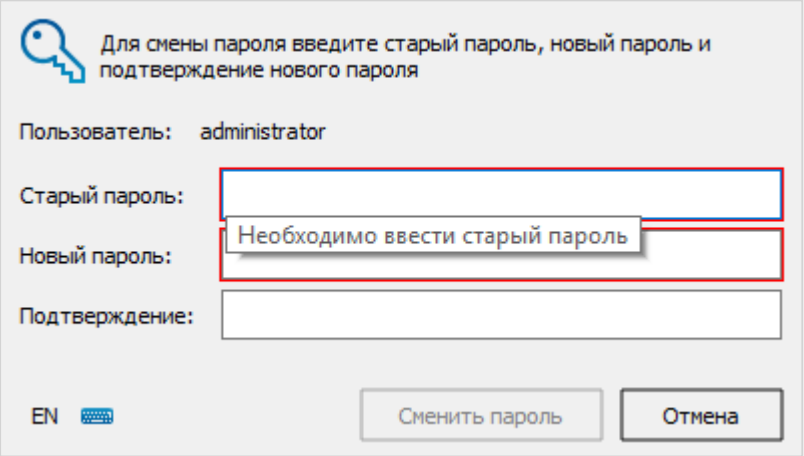
По умолчанию конфигуратор подключается к LDAP-серверу и каталогу, указанным в конфигурационном файле агента безопасности `alpha.security.agent.xml`, расположенном в:

- `C:\Program Files\Automiq\Alpha.Security` для ОС Windows;
- `/opt/Automiq/Alpha.Security` для ОС Linux.

Если каталог указан в файле, но не существует (так бывает при запуске конфигуратора после первой установки Alpha.Security, или в результате редактирования файла), то он будет создан автоматически при первом запуске конфигуратора.

Имя каталога, указанное в конфигурационном файле по умолчанию - **`Alpha.Security`**. Учетные данные администратора, обладающего правами на просмотр и изменение конфигурации:

- логин - **«`administrator`»**;
- пароль - такой же, какой был введен при установке LDAP-сервера в составе Alpha.Security (если этот шаг был пропущен, значение пароля по умолчанию - **«`secret`»**). Далее этот пароль нужно будет обновить в целях безопасности.



Учетную запись администратора, автоматически созданную вместе с каталогом, впоследствии можно будет заменить на другую и удалить.

1.6. Подключение к произвольному каталогу на текущем или удаленном LDAP-сервере

После успешной авторизации (стр. 10) по умолчанию конфигуратор подключается к LDAP-серверу и каталогу, указанным в конфигурационном файле агента безопасности `alpha.security.agent.xml`, расположенном в:

- `C:\Program Files\Automiq\Alpha.Security` - для ОС Windows;
- `/opt/Automiq/Alpha.Security` - для ОС Linux.

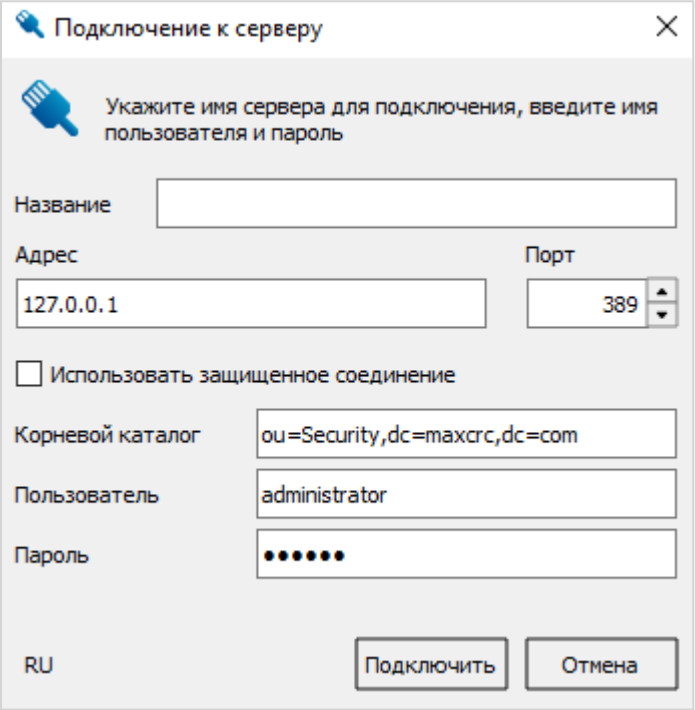
ПРИМЕР

```
<SecurityDn value="ou=Security,dc=maxcrc,dc=com"/>
```

Можно сразу перейти к редактированию конфигурации каталога, либо выполнить подключение к другому каталогу.

Для подключения к другому каталогу, в том числе, расположенному на удаленном LDAP-сервере, нажмите кнопку **Подключиться к серверу...** на панели инструментов конфигуратора. Откроется диалоговое окно, в котором можно указать:

- название подключения - указанное имя будет отображаться в списке подключений;
- адрес LDAP-сервера;
- порт (для защищенного подключения используется порт «636»);
- отметку об использовании защищенного подключения;
- каталог для подключения. Каталог указывается в формате «ou="имя-каталога",dc="домен-базы-данных"»;
- логин и пароль администратора каталога, к которому выполняется подключение.



Укажите необходимые данные и нажмите **Подключить**. Если ошибок не возникнет, в конфигураторе отобразится содержимое указанного каталога. Теперь можно перейти к редактированию конфигурации каталога, к которому выполнено подключение.

ОБРАТИТЕ ВНИМАНИЕ

Для подключения к удаленному LDAP-серверу понадобится предварительно настроить соединение текущего агента Alpha.Net с ним. Подробнее об этом - в документации для Alpha.Security.

ПРИМЕЧАНИЕ

Последние два подключения отображаются в списке подключений. Открыть список можно, нажав справа от кнопки **Подключиться к серверу...** на панели инструментов конфигуратора. По умолчанию список содержит только пункт **Локальный агент** - это подключение к корневому каталогу, указанному в конфигурационном файле `alpha.security.agent.xml`. Для удаления всех подключений, кроме локального, нажмите **Очистить**.

1.7. Редактирование конфигурации

Под редактированием конфигурации каталога, к которому выполнено подключение, подразумевается:

- 1.7.1. Создание и редактирование групп пользователей (стр. 13);
- 1.7.2. Создание приложений и прав (стр. 14);
- 1.7.3. Назначение прав (стр. 16) пользователям и группам;
- 1.7.4. Создание ролей (стр. 17);
- 1.7.5. Создание и редактирование учетных записей пользователей (стр. 18).

Также конфигуратор позволяет:

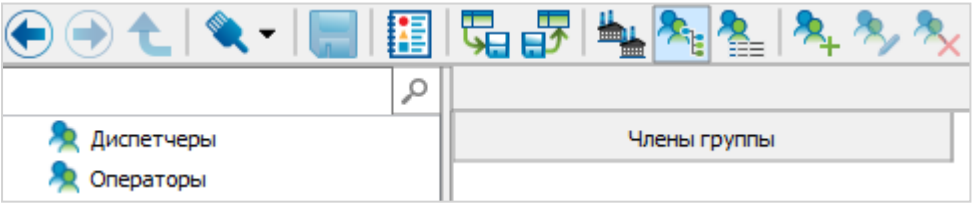
- создавать приложения посредством импорта из файла-шаблона приложения при настройке безопасности в других приложениях, являющихся компонентами Альфа платформы (стр. 23);
- создавать и восстанавливать резервные копии конфигурации каталога (стр. 25).

1.7.1. Создание и редактирование групп пользователей

Пользователей можно объединять в группы для удобства. Это помогает назначать одинаковые разрешения и запреты нескольким пользователям одновременно.
Создание и редактирование групп ведется в окне просмотра групп. Чтобы открыть окно, нажмите **Показать группы пользователей**.

Создание новой группы

Чтобы создать новую группу, нажмите **Добавить группу (Insert)** . В открывшемся окне введите название группы и нажмите **OK**. Группа появится в списке групп.



Редактирование группы

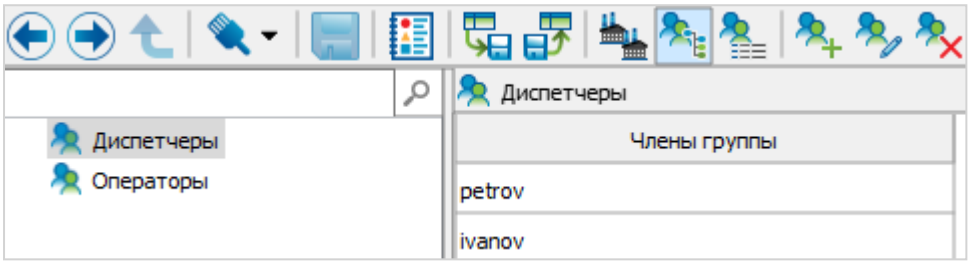
Чтобы редактировать группу, нажмите **Редактировать группу (Enter)** или дважды кликните по строке группы в списке.

Поиск группы в списке групп

Чтобы найти группу в списке групп по имени, воспользуйтесь панелью поиска над списком.

Сортировка списка пользователей в группе

Для сортировки списка пользователей, состоящих в группе, нажмите на заголовок списка **Члены группы**. Первое нажатие сортирует список по возрастанию, второе - по убыванию, третье нажатие отключает сортировку.



Специальные команды редактирования групп

Команда	Кнопка	Описание
Блокировать группу пользователей		Используется, когда необходимо ограничить доступ в систему всем пользователям, состоящим в группе. Пока группа заблокирована, попытки входа участников отклоняются подсистемой безопасности. Чтобы разблокировать заблокированную группу, нажмите . ОБРАТИТЕ ВНИМАНИЕ Блокировка группы, в которой находится текущий пользователь, невозможна.

1.7.2. Создание приложений и прав

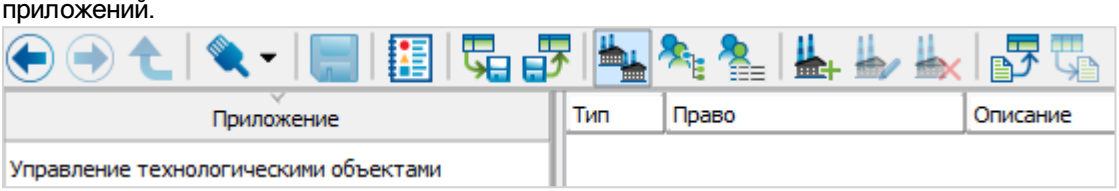
Возможности пользователей в проекте определяются наличием у них разрешений и запретов на определенные действия. Информация о том, разрешено или запрещено пользователю какое-либо действие, хранится в праве. Для удобства права сгруппированы в приложения. Подробнее об этом - в руководстве администратора подсистемы безопасности Alpha.Security.
Создание и редактирование приложений и прав ведется в окне просмотра приложений. Чтобы открыть окно, нажмите **Показать список приложений**.

Приложение можно создать, либо импортировать из файла (стр. 15).

Создание нового приложения

Чтобы создать новое приложение, нажмите **Добавить приложение (Insert)** на панели инструментов,

либо кликните правой кнопкой мыши в списке приложений и в открывшемся меню выберите пункт **Добавить приложение**. В открывшемся окне введите название приложения и нажмите **OK**. Приложение появится в списке приложений.



Добавление прав в приложение

Чтобы добавить в приложение права, нажмите **Редактировать приложение (Enter)** на панели

инструментов.

> ПРИМЕЧАНИЕ
>
> Перейти к редактированию приложения также можно:
>
> - кликнув по строке приложения в списке дважды;
> - выбрав строку приложения в списке и нажав клавишу Enter;
> - кликнув правой кнопкой мыши по строке приложения в списке, и в открывшемся меню выбрав пункт **Редактировать приложение**.

Откроется окно редактирования приложения. Для добавления, редактирования и удаления прав воспользуйтесь соответствующими кнопками на панели инструментов. После внесения изменений нажмите **Сохранить изменения** .

Экспорт и импорт приложений

В окне просмотра приложений доступен просмотр таблицы с информацией о правах приложения, выбранного из списка. Эту таблицу можно сохранить в файл. Для этого нажмите **Экспорт в файл...** на панели

инструментов, либо кликните правой кнопкой мыши по строке приложения в списке, и в открывшемся меню выберите пункт **Экспорт в файл...**. Откроется диалоговое окно, где будет предложено выбрать полный путь к папке для хранения файла, имя файла и его расширение: `*.csv` или `*.xlsx` (о создании и применении файлов с расширением `*.json` написано ниже). Укажите требуемые параметры и нажмите **Сохранить**.

Приложение вместе с добавленными в него правами и ролями можно сохранить в файл-шаблон `*.json`. Такой файл можно использовать для импорта приложения в конфигурацию Alpha.Security.

- Чтобы сохранить шаблон приложения, выберите нужное приложение из списка и нажмите **Экспорт в файл...** на панели инструментов, либо кликните правой кнопкой мыши по строке приложения и в

  открывшемся меню выберите пункт **Экспорт в файл...**. В открывшемся окне укажите путь к папке для хранения файла, имя файла и расширение `*.json`, а затем нажмите **Сохранить**. Файл появится в указанном расположении.

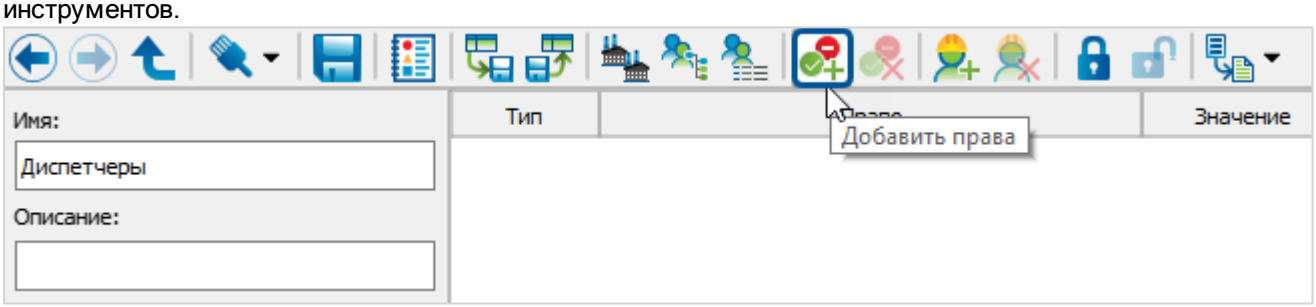
- Чтобы импортировать ранее сохраненный шаблон приложения, откройте окно просмотра приложений и нажмите **Импорт из файла...** на панели инструментов, либо кликните правой кнопкой мыши в списке приложений и в открывшемся меню выберите пункт **Импорт из файла...**. В открывшемся окне перейдите к расположению файла и откройте его. Приложение с его правами и ролями появится в списке приложений.

1.7.3. Назначение прав

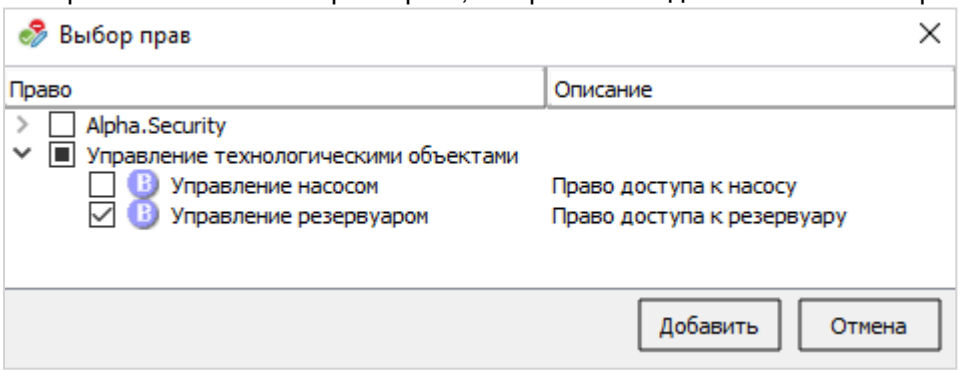
Право может быть назначено:

- пользователю лично - в окне редактирования учетной записи пользователя;
- группе пользователей - в окне редактирования группы;
- роли - в окне редактирования роли.

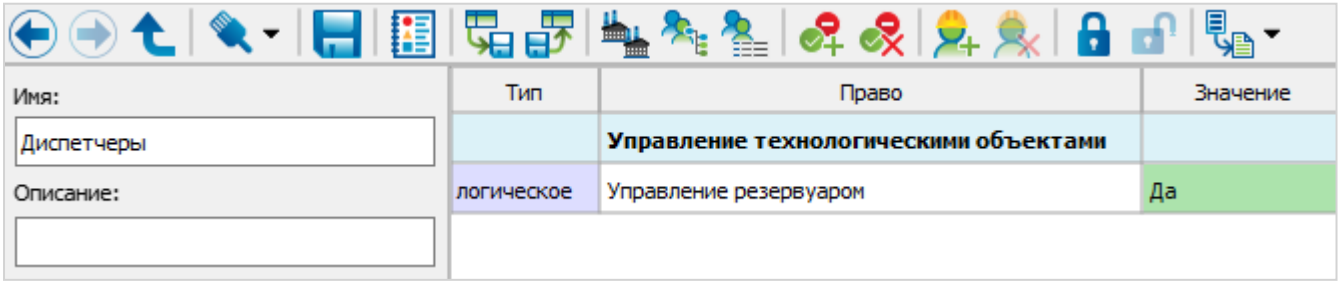
В результате эффективное значение права для пользователя зависит от того, какое значение права назначено ему лично, в каких группах он состоит и какие роли ему назначены. Подробнее об этом в 1.7.6. Вычисление эффективных значений прав (стр. 22) (стр. 22).
Далее описано, как назначить права группам. Пользователям и ролям права назначаются аналогично.
Чтобы назначить право группе, перейдите к ее редактированию. Нажмите **Добавить права** на панели инструментов.



В открывшемся окне выберите права, которые необходимо назначить выбранной группе и нажмите **Добавить**.



В списке прав, назначенных группе, появятся выбранные права. Чтобы изменить значение права, дважды кликните в ячейке столбца **Значение**.



После внесения всех изменений нажмите **Сохранить изменения** .

ПРИМЕЧАНИЕ

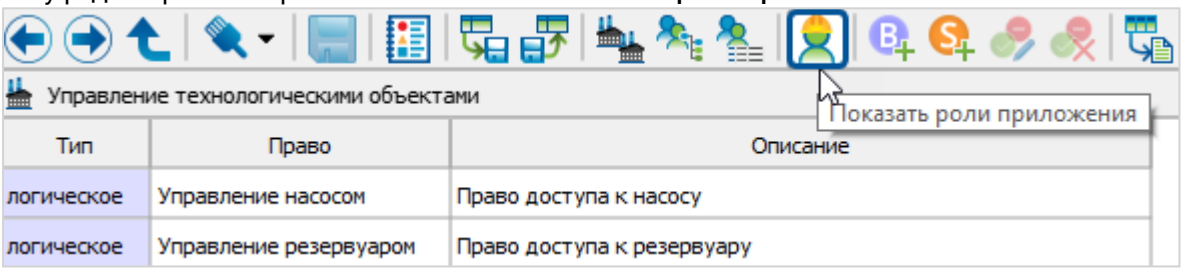
В окне выбора прав можно выбрать несколько прав одновременно.

ПРИМЕЧАНИЕ

Приложение **Alpha.Security** - это системное приложение, оно содержит системные права. Их можно назначить пользователю, например, чтобы разрешить просмотр и редактирование конфигурации, или ограничить длительность сессии пользователя. Подробнее каждое право описано в 1.9.1. Права системного приложения Alpha.Security (стр. 28).

1.7.4. Создание ролей

Внутри приложения можно создать роль. Роль - это совокупность значений каждого права приложения. Роль может быть назначена как пользователю, так и группе.
Создание и редактирование ролей ведется в окне со списком ролей. Чтобы открыть это окно, перейдите к окну редактирования приложения и нажмите **Показать роли приложения**.



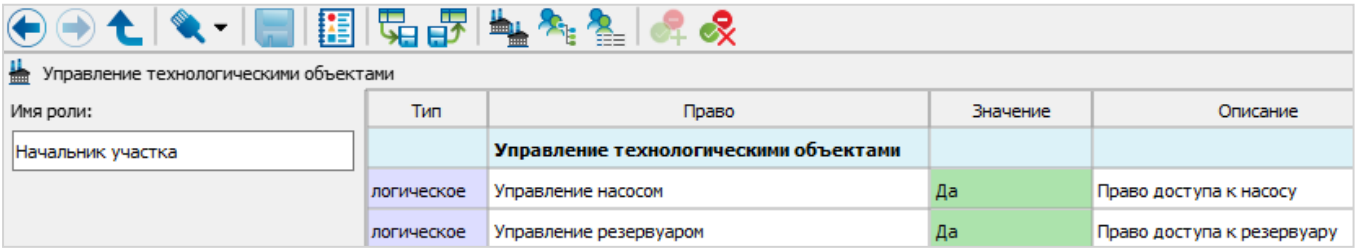
Создание новой роли

Чтобы создать новую роль:

1. В окне со списком ролей нажмите **Добавить роль (Insert)** на панели инструментов.
2. В открывшемся окне введите название роли и нажмите **OK**. Роль появится в списке ролей.

Назначьте созданной роли права текущего приложения с нужными значениями. Для этого:

1. Выделите ее в списке ролей и нажмите **Редактировать роль (Enter)**. Откроется окно редактирования выбранной роли.
2. В этом окне нажмите **Добавить права** на панели инструментов. Откроется окно **Выбор прав**.
3. В этом окне выберите нужные права и нажмите **Добавить**. Права будут добавлены роли, и для каждого добавленного права можно будет указать нужные значения. Чтобы изменить значение права, дважды кликните в ячейке столбца **Значение**.



После внесения изменений нажмите **Сохранить изменения** .

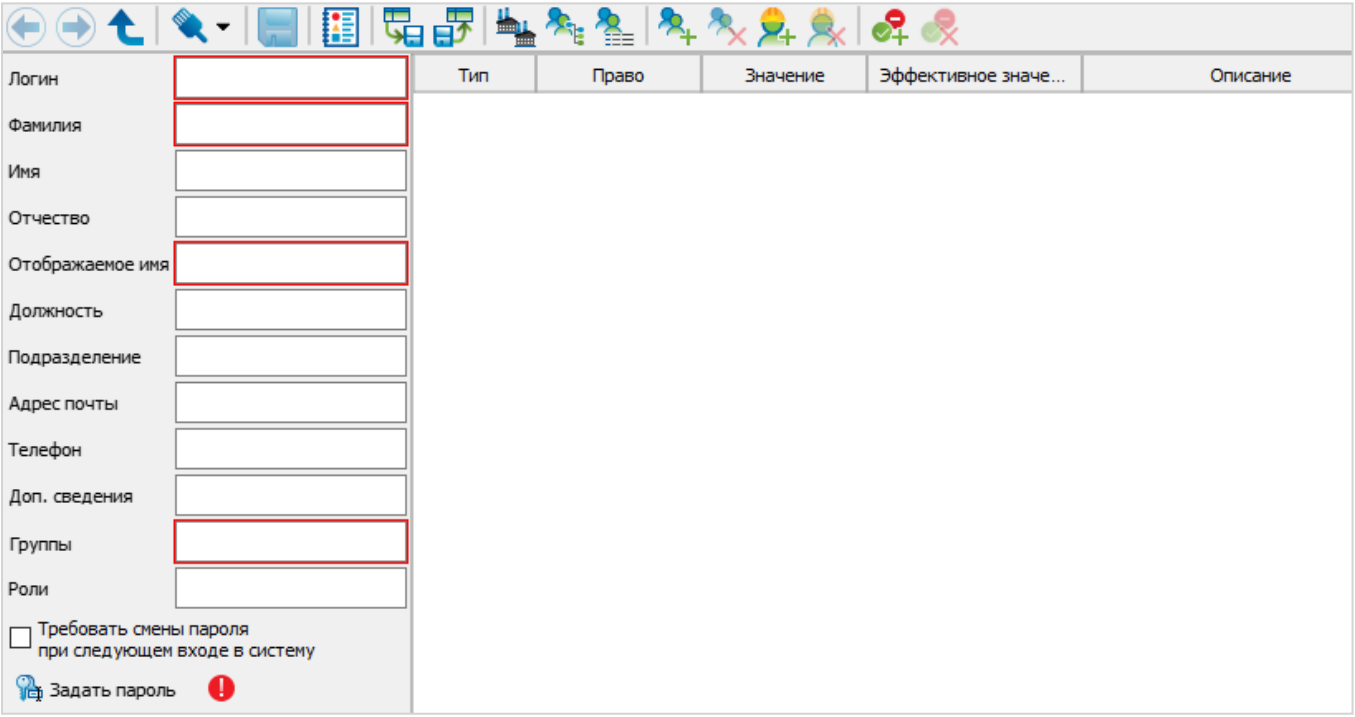
1.7.5. Создание и редактирование учетных записей пользователей

Чтобы пользователи могли получить доступ к возможностям проекта, каждому из них необходимо создать личную учетную запись.
Создание и редактирование списка учетных записей ведется в окне просмотра учетных записей. Чтобы открыть окно, нажмите **Показать список пользователей**.

Учетную запись для пользователя можно создать, либо импортировать из домена (стр. 19) или из внешней системы аутентификации (стр. 19).

Создание новой учетной записи

Чтобы создать новую учетную запись, нажмите **Добавить учетную запись пользователя (Insert)** на панели инструментов, либо кликните правой кнопкой мыши в списке пользователей и в открывшемся меню выберите пункт **Добавить учетную запись пользователя**. Откроется окно создания и редактирования учетной записи.



Заполните все необходимые поля, в том числе выделенные красной рамкой. Задайте пароль для новой учетной записи, используя кнопку **Задать пароль**. Кнопка отображается до тех пор, пока пароль не будет задан. Добавьте пользователя в группу, в противном случае сохранить учетную запись не удастся.
После внесения изменений нажмите **Сохранить изменения** .

Созданному пользователю не будут назначены никакие права.

> ПРИМЕЧАНИЕ
> Поле **Отображаемое имя** заполняется автоматически, его значение состоит из введенных фамилии, имени и отчества. Однако значение отображаемого имени можно менять.

ПРИМЕЧАНИЕ

Количество групп, в которые пользователь может быть добавлен одновременно, регулируется свойством **UserInOnlyOneGroup** компонента 2.2. Настройки (стр. 34) или настройками приложения (стр. 26).

Импорт учетных записей из домена

Alpha.HMI.SecurityConfigurator позволяет импортировать пользователей из домена.

Чтобы создать учетную запись для пользователя из домена, нажмите кнопку **Импорт пользователей из домена...** на панели инструментов. В верхней части открывшегося окна выберите группу, в которую будет добавлен пользователь, а затем отметьте его в списке.

ПРИМЕЧАНИЕ

Импортировать можно сразу нескольких пользователей.

После этого нажмите кнопку **Добавить**. В журнале, который можно открыть или скрыть по нажатию кнопки на панели инструментов, в процессе импорта будут появляться информационные сообщения о ходе этого процесса. В результате импорта появится диалоговое окно с сообщением об успешном импорте пользователя, и новая учетная запись появится в окне просмотра учетных записей.

Созданному пользователю не будут назначены никакие права.

Импорт учетных записей из внешних систем аутентификации

Alpha.HMI.SecurityConfigurator позволяет импортировать учетные записи из внешних источников (например, доменов).

ОБРАТИТЕ ВНИМАНИЕ

Источники должны быть заранее описаны в настройках агента безопасности. Подробнее об этом - в документации для Alpha.Security, раздел "Агент безопасности и его настройка", подраздел "Настроить подключение к внешним источникам учетных данных".

Чтобы импортировать учетную запись пользователя внешней системы, нажмите кнопку рядом с кнопкой **Импорт пользователей из домена...**, в открывшемся списке выберите команду **Импорт из внешней системы аутентификации...**. Откроется окно подключения к внешней системе. Укажите идентификатор системы (как в настройках агента безопасности), имя системы, учетные данные администратора этой системы, а затем нажмите **Подключить**. Если подключение будет выполнено успешно, откроется окно импорта учетных записей. В верхней части открывшегося окна выберите группу, в которую будет добавлен новый пользователь, а затем отметьте его в списке.

ПРИМЕЧАНИЕ

Импортировать можно сразу нескольких пользователей.

После этого нажмите кнопку **Добавить**. В журнале, который можно открыть или скрыть по нажатию кнопки на панели инструментов, в процессе импорта будут появляться информационные сообщения о ходе этого процесса. В результате импорта появится диалоговое окно с сообщением об успешном импорте пользователя, и новая учетная запись появится в окне просмотра учетных записей.

Созданному пользователю не будут назначены никакие права.

Редактирование учетных записей

Чтобы редактировать учетную запись, выберите ее в списке учетных записей, а затем нажмите **Редактировать учетную запись пользователя (Enter)** на панели инструментов.

ПРИМЕЧАНИЕ

Перейти к редактированию учетной записи также можно:

- кликнув по строке нужной учетной записи в таблице дважды;
- кликнув правой кнопкой мыши по строке нужной учетной записи в таблице и в открывшемся меню выбрав пункт **Редактировать учетную запись пользователя**.

Удаление учетных записей

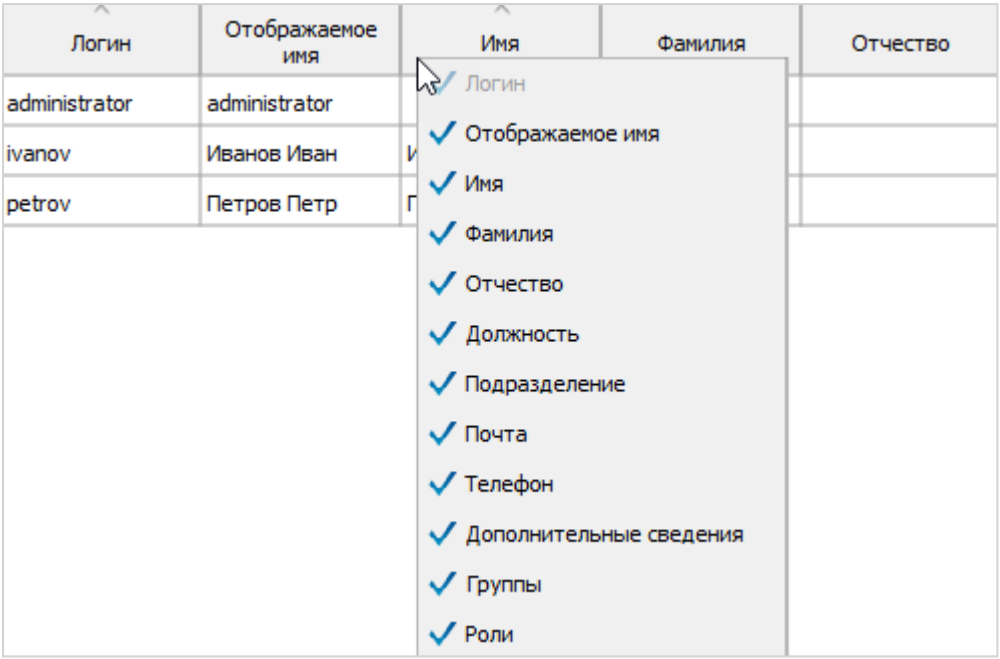
Чтобы удалить учетную запись пользователя, в окне просмотра пользователей выберите пользователя в таблице и нажмите кнопку **Удалить учетную запись пользователя (Delete)** на панели инструментов, либо кликните правой кнопкой мыши по строке нужной учетной записи в таблице и в открывшемся меню выберите пункт **Удалить учетную запись пользователя**.

Настройка отображения таблицы в окне просмотра учетных записей

В таблице, отображаемой в окне просмотра учетных записей, можно изменить набор столбцов и включить сортировку.

Чтобы изменить набор столбцов таблицы, выполните одно из следующих действий:

- кликните правой кнопкой мыши по заголовку таблицы и в контекстном меню отметьте нужные столбцы;
- кликните правой кнопкой мыши в окне просмотра учетных записей и в открывшемся меню выберите пункт **Столбцы**. В открывшемся меню отметьте нужные столбцы.



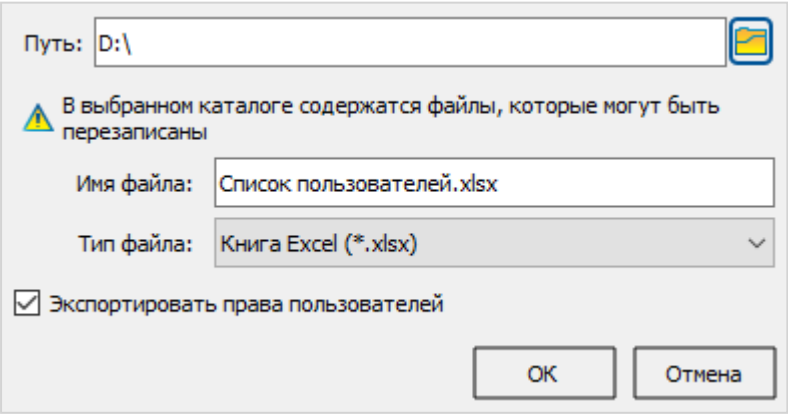
Для сортировки таблицы по содержимому одного из столбцов нажмите на заголовок этого столбца. Первое нажатие сортирует таблицу по содержимому столбца по возрастанию, второе - по убыванию, третье нажатие отключает сортировку.

Экспорт списка пользователей и списков их эффективных прав

Таблицу со списком пользователей можно сохранить в файл. Для этого можно:

- кликнуть правой кнопкой мыши в окне просмотра учетных записей и в контекстном меню выбрать **Экспорт в файл...**;
- нажать **Экспорт в файл...** на панели инструментов.

Откроется диалоговое окно, где будет предложено выбрать расположение, имя файла и его расширение: `*.csv` или `*.xlsx`. Здесь же можно указать, следует ли экспортировать списки прав каждого пользователя в отдельные файлы с таким же расширением, как у файла списка пользователей. Чтобы изменить расположение файла, нажмите **Выбрать папку**.



Укажите требуемые параметры и нажмите **OK**.

ПРИМЕЧАНИЕ

В файл со списком пользователей сохраняются те данные, которые отображаются в таблице в окне просмотра учетных записей (логин, имя, фамилия и пр.). Чтобы изменить набор данных в сохраняемом файле, измените видимость столбцов в таблице. Также можно изменить ширину столбцов.

Специальные команды редактирования учетных записей

Команда	Кнопка	Описание
Блокировать учетную запись пользователя		Используется, когда необходимо ограничить доступ пользователя в систему. Пока пользователь заблокирован, попытки входа отклоняются подсистемой безопасности. Чтобы разблокировать заблокированную учетную запись, нажмите . ОБРАТИТЕ ВНИМАНИЕ Блокировка учетной записи текущего пользователя невозможна.
Завершить сессию пользователя на указанных АРМ.		Предоставляет администратору возможность завершить сессии выбранного пользователя на указанных АРМ. Наличие команды связано со свойством DomainNodesList компонента `SecurityConfigurator`. Подробнее свойство описано в справочном руководстве (стр. 33).

1.7.6. Вычисление эффективных значений прав

Значение одного и того же права может быть назначено:

- пользователю лично;
- группе пользователей;
- роли.

Значение права для пользователя зависит от того, в каких группах он состоит, какие роли ему назначены, и какое значение права назначено пользователю лично. Итоговое значение называется эффективным значением права.

Правила определения эффективного значения права:

- Для логического права:
  - если есть хоть одно разрешающее значение и нет запрещающих, эффективное значение разрешающее;
  - если есть хоть одно запрещающее значение, эффективное значение запрещающее.
- Для строковых прав эффективное значение складывается из всех наследованных прав. Строковые права отображаются списком.
- Эффективные значения для системных прав Alpha.Security приведены в 1.9.1. Права системного приложения Alpha.Security (стр. 28).

1.7.7. Безопасность в компонентах Альфа платформы

Подсистема безопасности Alpha.Security дает возможность разграничивать возможности пользователей в некоторых компонентах Альфа платформы:

- Alpha.Alarms и Alpha.HMI.Alarms;
- Alpha.Trends и Alpha.HMI.Trends;
- Alpha.HMI.SecurityConfigurator;
- Alpha.HMI.IntegrityControl;
- Alpha.HMI.SetPoints;
- Alpha.HMI.Statistics;
- Alpha.HMI.ComtradeViewer;
- Alpha.HMI.AdminConsole;
- Alpha.HMI.SystemLogViewer;
- Alpha.DevStudio.

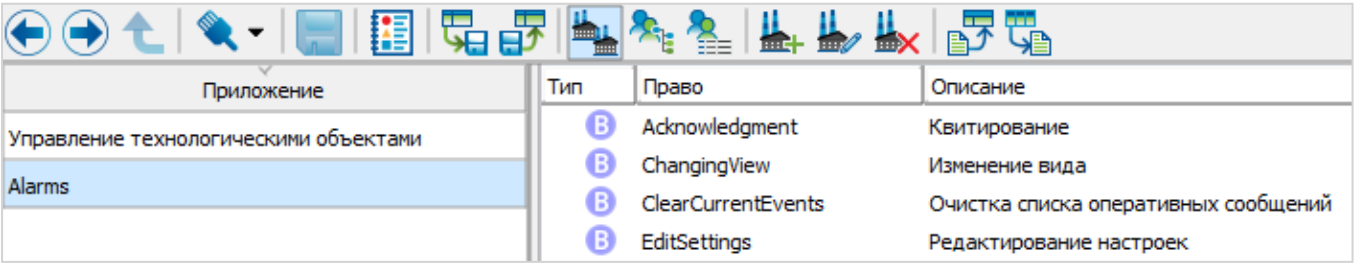
Подробнее о том, как именно активировать сервис безопасности, читайте в документах на соответствующие компоненты.
После того, как сервис безопасности активирован, необходимо назначить пользователям права на использование возможностей компонентов. Для этого необходимы приложения:

- **`Alarms`** для компонентов Alpha.Alarms и Alpha.HMI.Alarms;
- **`Trends`** для компонентов Alpha.Trends и Alpha.HMI.Trends;
- **`SecurityConfigurator`** для Alpha.HMI.SecurityConfigurator;
- **`IntegrityControl`** для Alpha.HMI.IntegrityControl;
- **`SetPoints`** для Alpha.HMI.SetPoints;
- **`Statistics`** для Alpha.HMI.Statistics;
- **`ComtradeViewer`** для Alpha.HMI.ComtradeViewer;
- **`AdminConsole`** для Alpha.HMI.AdminConsole;
- **`SystemLogViewer`** для Alpha.HMI.SystemLogViewer;
- **`DevStudio`** для Alpha.DevStudio.

Шаблоны этих приложений поставляются вместе с дистрибутивом Alpha.HMI.SecurityConfigurator и находятся в папке:

- `C:\ProgramData\Automiq\HMI.SecurityConfigurator\AppTemplates` для ОС Windows;
- `/usr/local/share/Automiq/HMI.SecurityConfigurator/AppTemplates` для ОС Linux.

Чтобы использовать нужное приложение в своих проектах, импортируйте его шаблон. Для этого откройте окно просмотра приложений и нажмите **`Импорт из файла...`** на панели инструментов, либо кликните правой кнопкой мыши в списке приложений и в открывшемся меню выберите пункт **`Импорт из файла...`**. Откроется окно импорта шаблонов приложений. Перейдите к расположению шаблонов, выберите нужный и нажмите **`Открыть`**. Приложение появится в списке приложений.



ПРИМЕЧАНИЕ

Путь к папке шаблонов приложений можно изменить в свойстве **AppTemplatesPath** компонента 2.2. Настройки (стр. 34) или в настройках приложения (стр. 26).

Для чего нужен шаблон SecurityConfigurator

Шаблон приложения **`SecurityConfigurator`** поставляет право **`Доступность файловой системы`**, позволяющее настроить доступ пользователей к файловой системе во время их работы с приложением Alpha.HMI.SecurityConfigurator. Импортируйте шаблон, чтобы назначить это право пользователям, имеющим доступ только к просмотру конфигурации, и которым необходимо ограничить доступ к файловой системе при экспорте файлов в окнах приложения Alpha.HMI.SecurityConfigurator. Также следует включить использование подсистемы безопасности Alpha.Security приложением в конфигурационном файле `app_settings.json`, подробнее об этом - в разделе 1.8. Настройки безопасности для разграничения возможностей пользователей (стр. 26).

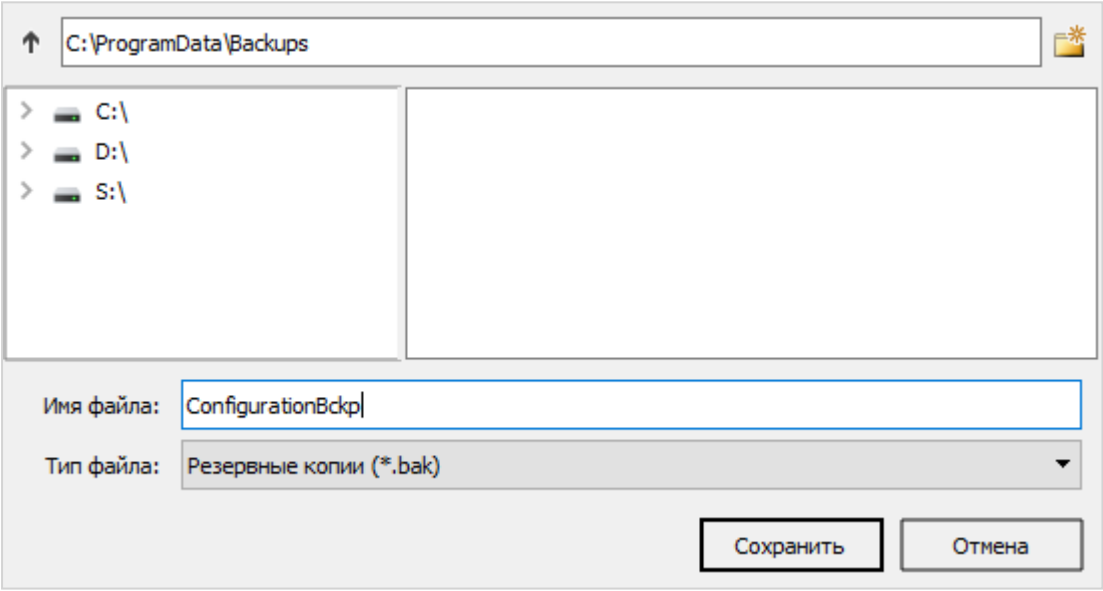
1.7.8. Резервное копирование конфигурации

Для сохранения текущей конфигурации подсистемы безопасности можно создать ее резервную копию. При необходимости сохраненную конфигурацию можно восстановить из резервной копии.

Создание резервной копии

Чтобы создать резервную копию текущей конфигурации, нажмите **Сохранить резервную копию конфигурации...**.

Откроется диалоговое окно, где будет предложено ввести полный путь к папке для хранения резервной копии и имя резервной копии. Укажите оба параметра и нажмите **Сохранить**.



- Чтобы перейти к папке на уровень выше, нажмите .
- Чтобы создать в указанном расположении новую папку, нажмите .

ОБРАТИТЕ ВНИМАНИЕ

Для создания резервной копии нужно обладать правами на чтение, создание и изменение файлов в указанной папке.

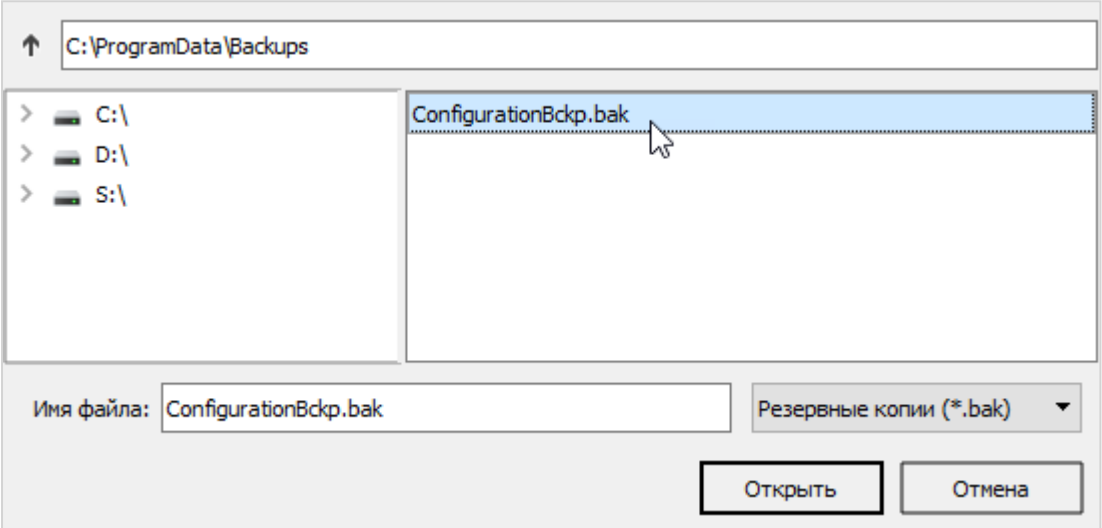
ПРИМЕЧАНИЕ

Путь к папке для хранения резервных копий конфигурации указывается в свойстве BackupsPath компонента **Настройки**. Если значение не указано, то по умолчанию используется путь к папке `Backups`, создающейся в папке проекта при первом сохранении резервной копии. Подробнее свойство описано в Справочном руководстве (стр. 35).

Восстановление из резервной копии

Чтобы восстановить конфигурацию из резервной копии, нажмите **Восстановить конфигурацию из резервной копии...**.

Откроется **Мастер восстановления конфигурации**, в котором будет предложено ввести полный путь к файлу резервной копии. В этом окне нажмите **Выбрать файл**, в открывшемся окне укажите полный путь к файлу резервной копии и полное имя резервной копии. Затем нажмите **Открыть**.



Выбранный путь к файлу будет отображен в мастере. В мастере нажмите **Выполнить**.

ОБРАТИТЕ ВНИМАНИЕ

Для восстановления резервной копии нужно обладать правами на чтение файлов в указанной папке.

1.8. Настройки безопасности для разграничения возможностей пользователей

Можно разграничить возможности пользователей:

- в окне приложения Alpha.HMI.SecurityConfigurator;
- в окне собственного приложения, разработанного на основе элементов расширений Alpha.HMI.SecurityConfigurator и Alpha.HMI.CommonLib, и предназначенного для редактирования конфигурации каталогов LDAP-сервера.

Пока под разграничением возможностей подразумевается только настройка доступа к файловой системе пользователей, имеющих разрешения на просмотр конфигурации.

Добавление прав в Alpha.Security

Доступ к файловой системе можно настроить, используя подсистему безопасности Alpha.Security. Необходимые для этого права описаны в таблице в конце этого раздела. Их можно создать самостоятельно, либо загрузить из шаблона приложения, поставляемого вместе с дистрибутивом конфигуратора. Для этого в

окне просмотра приложений в конфигураторе следует нажать кнопку импорта из файла. В открывшемся окне указать путь к папке с шаблонами приложений:

- `C:\ProgramData\Automiq\HMI.SecurityConfigurator\AppTemplates` для ОС Windows;
- `/usr/local/share/Automiq/HMI.SecurityConfigurator/AppTemplates` для ОС Linux.

Импортируйте шаблон приложения **`SecurityConfigurator`**. Приложение появится в текущей конфигурации подсистемы безопасности. Назначьте права приложения нужным пользователям, указав разрешающие или запрещающие значения прав.

Настройка безопасности в окне самостоятельного приложения Alpha.HMI.SecurityConfigurator

Чтобы включить использование подсистемы безопасности приложением:

1. В конфигурационном файле `app_settings.json` измените значение свойства `UseSecurity` объекта `Permissions` с **«false»** на **«true»**. Файл расположен в:
   - `C:\Program Files\Automiq\Alpha.HMI.SecurityConfigurator` для ОС Windows;
   - `/home/<user>/Automiq/HMI.SecurityConfigurator` для ОС Linux.
2. Сохраните файл и перезапустите приложение.

Теперь доступность файловой системы для пользователя будет зависеть от наличия у него прав из таблицы в конце этого раздела.

Настройка безопасности в окне собственного приложения

Предположим, вы разрабатываете собственный конфигуратор. В проекте такого приложения можно использовать либо тип 2.1. SecurityConfigurator (стр. 33), либо одну из форм 2.6. SecurityConfigurator_Form (стр. 37) или 2.5. SecurityConfigurator_Page (стр. 36).

Пусть необходимо ограничить доступ пользователей к файловой системе. Для этого необходимо добавить в проект экземпляр типа 2.3. Разрешения (стр. 35), поставляемого в составе модуля Alpha.HMI.SecurityConfigurator. У этого экземпляра есть свойство **Доступность файловой системы**, значение которого можно изменять. Значения свойства могут быть установлены вами, а могут вычисляться на основе значений прав текущего пользователя в подсистеме безопасности Alpha.Security.

Чтобы получать значения прав текущего пользователя, необходимо выполнить следующие шаги.

1. Укажите для созданного экземпляра типа 2.3. Разрешения (стр. 35) в качестве значения свойства **Использовать сервер безопасности для определения прав** значение **«true»**, а в качестве значения свойства **Имя приложения с правами безопасности** - название приложения, созданного вами в текущей конфигурации Alpha.Security.
2. Для типа или формы, используемой в проекте, в качестве значения свойства **Разрешения** укажите ссылку на созданный вами экземпляр типа 2.3. Разрешения (стр. 35).

Теперь доступность файловой системы для пользователя будет зависеть от наличия у него прав из таблицы в конце этого раздела.

Права из шаблона приложения SecurityConfigurator

Право	Назначение
Доступность файловой системы (FileSystemAccess)	Значение права регулирует доступность файловой системы для конкретного пользователя (или группы). При входе пользователя в подсистему безопасности свойство **Доступность файловой системы** экземпляра типа Разрешения принимает значение, с которым это право назначено текущему пользователю.

1.9. Приложения

1.9.1. Права системного приложения Alpha.Security

Право	Краткое описание	Описание
ActivityLimitedToOneWorkstationOnly	Пользователь может быть активен только на одном ARM	Используется, чтобы пользователь мог быть активен не более чем на одном АРМ. Используйте вместе с правом **ForceStopOldUserSessionOnNewLogin** для принудительного завершения активной сессии пользователя, если он выполняет вход на новом АРМ. Этим правом автоматически наделяется администратор текущего каталога LDAP. При создании новой учетной записи администратора следует назначить ему это право со значением **Нет**.
AttempsTimeOut	Таймаут блокировки при превышении количества неуспешных попыток входа, мин	Длительность блокировки пользователя при превышении количества неудачных попыток, указанных в **MaxAttempsCount**. Эффективным значением права является максимальное значение.
ConfigurationAccess	Редактирование конфигурации	Предоставляет доступ к редактированию конфигурации Alpha.Security. Этим правом автоматически наделяется администратор текущего каталога LDAP. При создании новой учетной записи администратора следует назначить ему это право со значением **Да**.
EditSettings	Изменение настроек	Предоставляет доступ к настройкам Alpha.HMI.SecurityConfigurator. Не применяется в текущей версии Alpha.HMI.SecurityConfigurator.

Право	Краткое описание	Описание
ForceStopOldUserSessionOnNewLogin	Завершать сессию на активном ARM при входе пользователя на другом ARM	Используется для принудительного завершения активной сессии пользователя, если он выполняет вход на новом АРМ. Этим правом автоматически наделяется администратор текущего каталога LDAP. При создании новой учетной записи администратора следует назначить ему это право со значением **Да**.
InteractiveLogon	Интерактивный вход	Используется для установки разрешения или запрета на вход. Этим правом автоматически наделяется администратор текущего каталога LDAP. При создании новой учетной записи администратора следует назначить ему это право со значением **Да**. Вместе с тем позволяет исключить учетную запись из списка пользователей, предоставляемого по запросу. Например: › из выпадающего списка пользователей в окне входа в конфигураторе; › из списка, запрашиваемого компонентом **Список пользователей** расширения Alpha.HMI.Security.
LowerCount	Количество в пароле символов в нижнем регистре	Устанавливает минимально допустимое количество символов в нижнем регистре в пароле. Эффективным значением права является максимальное значение.
MaxAttempsCount	Количество попыток входа, шт	Устанавливает количество неудачных попыток входа для пользователя. Если пользователь не войдет за указанное количество попыток, то блокируется на время, указанное в **AttempsTimeOut**. При вводе неправильного пароля Агент Alpha.Security сообщает о количестве оставшихся попыток входа до временной блокировки. Эффективным значением права является минимальное значение.

Право	Краткое описание	Описание
`MaxIdleTime`	Максимальное время бездействия, мин	Устанавливает время бездействия пользователя. Таймер сбрасывается при каждом взаимодействии пользователя с АРМ - щелчком или движением мыши, вводом текста с клавиатуры и т.д. Если же за указанное время пользователь не взаимодействует с АРМ, происходит автоматический выход пользователя из системы. Эффективным значением права является минимальное значение.
`NonDeletableWhileActive`	Запрет удаления активного пользователя	Устанавливает запрет на удаление учетной записи, если пользователь активен. Этим правом автоматически наделяется администратор текущего каталога LDAP. При создании новой учетной записи администратора следует назначить ему это право со значением **Да**.
`NumbersCount`	Количество цифровых символов в пароле	Устанавливает минимально допустимое количество цифр в пароле. Эффективным значением права является максимальное значение.
`PasswordAge`	Срок действия пароля, дней	Устанавливает границы срока действия пароля. До истечения минимального срока действия обновить пароль нельзя. После истечения максимального срока действия пароля попытки входа со старыми учетными данными будут отклоняться. Эффективным значением минимального срока является максимальное значение. Эффективным значением максимального срока является минимальное значение.
`PasswordComplexity`	Сложность пароля	Обязательность использования в пароле следующих видов символов: › цифры; › буквы нижнего регистра; › буквы верхнего регистра; › специальные символы. Эффективным значением является сочетание всех указанных символов.

Право	Краткое описание	Описание
PasswordComposition	Состав пароля	Позволяет ограничить набор доступных символов для пароля: только цифры, только символы нижнего или верхнего регистра, только символы, либо сочетание перечисленных ограничений.
PasswordMinLength	Минимальная длина пароля	Устанавливает минимально допустимое количество символов в пароле. Эффективным значением права является максимальное значение.
PasswordNotContainUserData	Проверка наличия пользовательских данных в пароле	Проверяет наличие логина, имени, фамилии и/или отображаемого имени пользователя в его пароле. Проверка выполняется без учета регистра. Эффективным значением права является **Нет** (установлен по умолчанию). В этом случае пароль может содержать пользовательские данные.
PasswordNotifyForChange	Уведомление о смене пароля, дней	Позволяет создать напоминание об истечении срока действия пароля для пользователя. За указанное до истечения пароля время будет отправлено напоминание о скором истечении срока действия пароля. Эффективным значением является максимальное значение.
PasswordsInHistory	Количество паролей в истории	Устанавливает количество хранимых в истории паролей. Обновить пароль на такой же, как в истории паролей, не получится. Эффективным значением является максимальное значение.
SessionDurationLimit	Максимальное время сессии, мин	Устанавливает длительность сессии пользователя. После истечения указанного времени происходит автоматический выход пользователя из системы. Эффективным значением является минимальное значение.

Право	Краткое описание	Описание
SpecialCount	Количество специальных символов в пароле	Устанавливает минимально допустимое количество специальных символов в пароле. **Специальные символы** ? ! @ # $ % ^ & № < > _ - = + * ( ) [ ] { } . , : ; ~ ` ' " \ | / Эффективным значением права является максимальное значение.
UpperCount	Количество в пароле символов в верхнем регистре	Устанавливает минимально допустимое количество символов в верхнем регистре в пароле. Эффективным значением права является максимальное значение.
UserSessionHoldTimeout	Время сохранения сессии пользователя при разрыве связи агента безопасности с хранилищем, сек	Устанавливает время сохранения сессии пользователя при разрыве связи агента безопасности с LDAP-сервером. Принимает значения в диапазоне от **«0»** до **«86400»**. Если значение не задано или равно **«0»**, то при разрыве связи сессия не будет сохранена. Значение по умолчанию: **«60»**. Эффективным значением права является минимальное значение.
ViewConfiguration	Просмотр конфигурации	Предоставляет доступ к просмотру конфигурации Alpha.Security без возможности редактирования. Этим правом автоматически наделяется администратор текущего каталога LDAP. При создании новой учетной записи администратора следует назначить ему это право со значением **Да**.
WinKeysShortcutAccess	Доступ к сочетаниям клавиш Windows	Блокирует использование сочетаний клавиш (так называемых Hotkeys). Чтобы запретить использование сочетаний клавиш, укажите их в конфигурационном файле Агент Alpha.Security. Подробнее об этом в документе на Alpha.Security, в разделе, описывающем настройку агента.

2. Справочное руководство

2.1. SecurityConfigurator

`SecurityConfigurator` - тип, предназначенный для размещения на форме в проекте Alpha.HMI.

Тип не является полноценной заменой приложения Alpha.HMI.SecurityConfigurator: экземпляр типа отличается от приложения тем, что в нем отсутствует верхняя панель инструментов и нижняя строка статуса.

Тип, используемый в проекте Alpha.HMI, в режиме рантайма создает файлы, описанные в таблице ниже.

Файл	Расположение	Назначение
`settings.json`	› `C:\ProgramData\Automiq\HMI.SecurityConfigurator` для ОС Windows; › `/home/<user>/Automiq/HMI.SecurityConfigurator` для ОС Linux.	Хранит информацию о папке экспорта файлов. Используется для указания папки, открываемой при экспорте файлов по умолчанию.
`session.json`	› `C:\Users\<user>\Automiq\HMI.SecurityConfigurator` для ОС Windows; › `/home/<user>/Automiq/HMI.SecurityConfigurator` для ОС Linux.	Хранит информацию об окне конфигуратора: размер окон и таблиц, последнюю открытую папку импорта файлов-шаблонов приложений, фильтры и сортировки списков и таблиц и т.д.. Файл используется для работы типа, менять его содержимое не рекомендуется.

2.1.1. Свойства

Error

Поле с текстом ошибки приложения. Только для чтения в режиме исполнения. Тип значения - уведомляющий string.

Status

Поле с текстом статуса приложения. Только для чтения в режиме исполнения. Тип значения - уведомляющий string.

CurrentForm

Хранит код статуса открытого во фрейме окна приложения (окно просмотра приложений, окно редактирования пользователя и т.п.). Только для чтения в режиме исполнения. Тип значения - уведомляющий uint1.

pClosed

Ссылка на команду, выполняемую в результате закрытия окна в экземпляре типа.

Настройки (init_Settings)

Ссылка на экземпляр типа 2.2. Настройки (стр. 34), хранящий настройки конфигуратора.

Разрешения (init_Permissions)

Ссылка на экземпляр типа 2.3. Разрешения (стр. 35), хранящий настройки доступа пользователей к файловой системе в диалогах сохранения файлов.

Вызов экранной клавиатуры (init_ScreenKeyboard)

Ссылка на экземпляр типа **`ScreenKeyboardOptions`**, хранящий настройки вызова экранной клавиатуры. Тип предоставляется библиотекой Alpha.HMI.CommonLib.

2.2. Настройки

Тип предназначен для хранения настроек конфигуратора.

Все экземпляры конфигуратора, сославшиеся на экземпляр этого типа, будут иметь одинаковые настройки.

Настройки описаны в свойствах типа.

2.2.1. Свойства

Пользователь должен быть в группе (UserInAtLeastOneGroup)

Регулирует обязательность добавления пользователя в группу. Тип значения - bool.

> ОБРАТИТЕ ВНИМАНИЕ
> Требование обязательного нахождения пользователя в группе теперь исходит из подсистемы безопасности Alpha.Security.

Значение:

- **`«true»`** - пользователь должен состоять хотя бы в одной группе. Пока пользователь не будет добавлен в группу, сохранить учетную запись не удастся, так как кнопка сохранения будет неактивна.
- **`«false»`** - пользователя необязательно добавлять в группу (группы), кнопка сохранения активна. Однако попытка сохранить учетную запись приводит к ошибке.

Пользователь может быть только в одной группе (UserInOnlyOneGroup)

Регулирует количество групп, в которых пользователь может состоять одновременно. Тип значения - bool.

Значение:

- **«true»** - пользователя можно добавить только в одну группу;
- **«false»** - пользователя можно добавить в несколько групп.

Путь к шаблонам приложений (AppTemplatesPath)

Полный путь к папке для хранения шаблонов приложений. Тип значения - string.

Если не указывать значение свойства, то при импорте приложений будет предложен путь к папке по умолчанию. Папкой по умолчанию является папка `Security_Templates` в папке проекта.

Путь к резервным копиям базы (BackupsPath)

Полный путь к папке для хранения резервных копий конфигурации подсистемы безопасности. Тип значения - string.

Если не указывать значение свойства, то при сохранении резервной копии или восстановлении из резервной копии будет предложен путь к папке по умолчанию. Папкой по умолчанию является папка `Backups`, создающаяся в папке проекта при первом сохранении резервной копии.

Папка для экспорта (ExportFolder)

Хранит полный путь к папке, куда пользователь может выполнять экспорт списков пользователей и шаблонов приложений в файлы. Тип значения - string.

По умолчанию списки параметров сохраняются в:

- `C:\ProgramData\Automiq\HMI.SecurityConfigurator\Export` - для Windows;
- `/home/<user>/Automiq/HMI.SecurityConfigurator/Export` - для Linux.

Список узлов домена (DomainNodesList)

Список имен АРМ, на которых можно завершить сессию пользователя. Тип значения - string.

Если указать хотя бы одно имя АРМ в сети Alpha.Net, в окне редактирования учетной записи пользователя появится команда на завершение сессии. Результат применения свойства описан в Специальные команды редактирования учетных записей (стр. 22)).

ПРИМЕР

Построена простейшая сеть Alpha.Net, в которой все узлы подчинены центральному узлу сети. В сеть включены три АРМ: **CentralNode**, **Node1** и **Node2**. Необходимо иметь возможность завершать пользовательские сессии на узлах **Node1** и **Node2** из конфигуратора на центральном узле **CentralNode**. Тогда в качестве значения свойства DomainNodesList указывается список имен **Node1** и **Node2**.

2.3. Разрешения

Тип, позволяющий управлять доступом пользователей к файловой системе в диалогах сохранения файлов. Если ограничить доступ, пользователь сможет экспортировать файлы только в папку, указанную в экземпляре типа 2.2. Настройки (стр. 34).

Для определения прав доступа текущего пользователя к файловой системе можно использовать конфигурацию подсистемы безопасности Alpha.Security. Подробнее - в описании свойств типа.

2.3.1. Свойства

Использовать сервер безопасности для определения прав (UseSecurity)

Позволяет включить использование подсистемы безопасности Alpha.Security для определения прав доступа текущего пользователя к файловой системе.

Значения:

- **«true»** - использовать сервер безопасности для определения прав;
- **«false»** - не использовать сервер безопасности для определения прав.

При включении следует заполнить значение свойства **Имя приложения с правами безопасности (SecurityApplication)**.

Доступность файловой системы (FileSystemAccess)

Позволяет включить или отключить доступ пользователя к файловой системе в диалогах сохранения файлов.

Значения:

- **«true»** - файловая система доступна, в диалогах сохранения файлов есть доступ к любой папке;
- **«false»** - файловая система недоступна. Пользователь может сохранять файлы только в папку, указанную в свойстве **Папка для экспорта (ExportFolder)** экземпляра типа 2.2. Настройки (стр. 34).

Имя приложения с правами безопасности (SecurityApplication)

Хранит имя приложения Alpha.Security, в котором содержится право **FileSystemAccess (Доступность файловой системы)**, регулирующее доступность файловой системы для конкретного пользователя или группы пользователей.

Чтобы использовать сервер безопасности для определения прав текущего пользователя, установите значение **«true»** для свойства **Использовать сервер безопасности для определения прав (UseSecurity)**.

2.4. SecurityConfigurator_App

SecurityConfigurator_App - форма, используемая для открытия полноценного приложения Alpha.HMI.SecurityConfigurator из меню Пуск.

Не имеет собственных свойств и функций.

2.5. SecurityConfigurator_Page

Экранная форма **SecurityConfigurator_Page** предназначена для открытия окна конфигуратора во фрейме. В режиме исполнения позволяет:

- создавать, редактировать и удалять учетные записи пользователей;
- создавать, редактировать и удалять группы пользователей;
- добавлять пользователей в группы и удалять их из групп;
- создавать, редактировать и удалять приложения и права, назначать права пользователям и группам;
- создавать роли в приложениях и назначать их пользователям и группам;
- сохранять резервные копии конфигурации и восстанавливать конфигурацию из резервной копии.

2.5.1. Свойства

Тема оформления (init_Theme)

Ссылка на экземпляр типа **Theme**, хранящий настройки оформления конфигуратора. Тип предоставляется библиотекой Alpha.HMI.CommonLib.

Настройки (init_Settings)

Ссылка на экземпляр типа 2.2. Настройки (стр. 34), хранящий настройки конфигуратора.

Разрешения (init_Permissions)

Ссылка на экземпляр типа 2.3. Разрешения (стр. 35), хранящий настройки доступа пользователей к файловой системе в диалогах сохранения файлов.

Вызов экранной клавиатуры (init_ScreenKeyboard)

Ссылка на экземпляр типа **ScreenKeyboardOptions**, хранящий настройки вызова экранной клавиатуры. Тип предоставляется библиотекой Alpha.HMI.CommonLib.

2.6. SecurityConfigurator_Form

Экранная форма **SecurityConfigurator_Form** предназначена для открытия окна конфигуратора в отдельном окне. В режиме исполнения позволяет:

- создавать, редактировать и удалять учетные записи пользователей;
- создавать, редактировать и удалять группы пользователей;
- добавлять пользователей в группы и удалять их из групп;
- создавать, редактировать и удалять приложения и права, назначать права пользователям и группам;
- создавать роли в приложениях и назначать их пользователям и группам;
- сохранять резервные копии конфигурации и восстанавливать конфигурацию из резервной копии.

Форма, используемая в проекте Alpha.HMI, в режиме рантайма создает файл `window.json` в папке:

- `C:\Users\<user>\Automiq\HMI.SecurityConfigurator` для ОС Windows;
- `/home/<user>/Automiq/HMI.SecurityConfigurator` для ОС Linux.

Файл используется для работы формы, менять его содержимое не рекомендуется.

2.6.1. Свойства

Тема оформления (init_Theme)

Ссылка на экземпляр типа **Theme**, хранящий настройки оформления конфигуратора. Тип предоставляется библиотекой Alpha.HMI.CommonLib.

Настройки (init_Settings)

Ссылка на экземпляр типа 2.2. Настройки (стр. 34), хранящий настройки конфигуратора.

Разрешения (init_Permissions)

Ссылка на экземпляр типа 2.3. Разрешения (стр. 35), хранящий настройки доступа пользователей к файловой системе в диалогах сохранения файлов.

Восстанавливать положение окна (init_RestoreWindow)

Позволяет восстанавливать положение окна после его повторного открытия. Тип значения - bool

Значения:

- **«true»** - положение окна запоминается. Окно откроется в том же месте, где было закрыто.
- **«false»** - положение окна не запоминается. Окно откроется в координатах по умолчанию.

Вызов экранной клавиатуры (init_ScreenKeyboard)

Ссылка на экземпляр типа **ScreenKeyboardOptions**, хранящий настройки вызова экранной клавиатуры. Тип предоставляется библиотекой Alpha.HMI.CommonLib.

2.7. SecurityConfigurator_Tests

Форма предназначена для ознакомления с примерами использования форм расширения 2.6. SecurityConfigurator_Form (стр. 37) и 2.5. SecurityConfigurator_Page (стр. 36) в проекте Alpha.HMI.

Чтобы ознакомиться с этими примерами, откройте проект расширения в режиме редактирования, а затем перейдите к форме.

Форма, используемая в проекте Alpha.HMI, в режиме рантайма создает файл `tests.json` в папке:

- `C:\Users\<user>\Automiq\HMI.SecurityConfigurator` для ОС Windows;
- `/home/<user>/Automiq/HMI.SecurityConfigurator` для ОС Linux.

Файл используется для работы формы, менять его содержимое не рекомендуется. Чтобы ознакомиться с тем, как создавать подобные файлы для восстановления внешнего вида и положения окна формы, откройте проект расширения в режиме редактирования и ознакомьтесь с обработчиками событий формы на вкладке **События**.

История изменений

3.0

Улучшения

- Добавлена возможность использования новых системных прав: **Состав Пароля**, **Проверка наличия пользовательских данных в пароле**, **Время сохранения сессии пользователя при разрыве связи агента безопасности с хранилищем, сек**.
- На панель инструментов добавлены кнопки **Назад**, **Вперед**, **Наверх**.
- Изменен внешний вид окна просмотра приложений и окна редактирования ролей.
- Формат иконок приложения изменен на SVG. Ранее использовался формат PNG.
- Изменены расположение и содержимое конфигурационных файлов приложения:
  - Настройки приложения и компонентов расширения хранятся в файлах `app_settings.json` и `settings.json`, расположенных в папке:
    - `C:\ProgramData\Automiq\HMI.SecurityConfigurator` для ОС Windows;
    - `/home/<user>/Automiq/HMI.SecurityConfigurator` для ОС Linux.
  - Настройки сессии приложения хранятся в файле `app_session.json`, настройки сессии компонентов - в файле `session.json`. Эти файлы расположены в папке:
    - `C:\Users\<user>\Automiq\HMI.SecurityConfigurator` для ОС Windows;
    - `/home/<user>/Automiq/HMI.SecurityConfigurator` для ОС Linux.

Исправления

- При удаленном подключении к LDAP-серверу администратору была доступна кнопка блокирования его учетной записи. В таком случае единственный администратор, заблокировав свою учетную запись, мог полностью заблокировать доступ к каталогу.
- Исправлено блокирование кнопок панели инструментов при повторном подключении к удаленному LDAP-серверу.
- Исправлено сообщение о разрыве соединения с LDAP-сервером. Ранее ошибочно появлялось сообщение об отсутствии прав на просмотр конфигурации у текущего пользователя.
- В диалоговых окнах сохранения файлов кнопки **Скрыть папки** и **Вызвать экранную клавиатуру** теперь отображаются корректно.

Изменения документации

Редакция 1

В руководстве пользователя:

- Актуализирован раздел 1.3. Запуск самостоятельного приложения (стр. 7).
- Бывший раздел "Настройка приложения в конфигурационных файлах" переименован в 1.8. Настройки безопасности для разграничения возможностей пользователей (стр. 26) и расширен: здесь описаны настройки безопасности как для самостоятельного приложения Alpha.HMI.SecurityConfigurator, так и для собственных приложений, разрабатываемых с помощью компонентов расширения.
- Актуализирована информация в 1.9.1. Права системного приложения Alpha.Security (стр. 28).
- Актуализированы скриншоты.

Также актуализировано справочное руководство: для каждого компонента расширения описано, какие конфигурационные файлы могут быть сгенерированы при его использовании, и для чего они предназначены.

2.5

Новые возможности

- Добавлена возможность подключения к произвольному LDAP-серверу. Для этого на панель инструментов добавлена кнопка **Подключиться к серверу**, нажатие на которую открывает диалоговое окно подключения, в котором необходимо указать адрес и порт нужного LDAP-сервера, название корневого каталога и учетные данные администратора этого каталога.
- Добавлена возможность импорта учетных записей из внешних источников учетных данных. Для этого рядом с кнопкой импорта учетных записей добавлена кнопка, открывающая выпадающий список, содержащий команду открытия окна подключения к внешнему источнику данных. Источники должны быть заранее описаны в настройках агента безопасности. Для подключения ко внешнему источнику необходимо указать его идентификатор (как в настройках агента безопасности), имя, а так же учетные данные администратора этого источника.

Изменения

- Изменено окно восстановления конфигурации из резервной копии. Теперь если восстановление конфигурации выполняется для удаленного LDAP-сервера, потребуется указать учетные данные администратора этого LDAP-сервера.
- Срок действия пароля теперь можно увеличивать до 750 дней.

Изменения документации

Редакция 1

Изменена структура руководства пользователя: разделы, описывающие непосредственно изменение конфигурации каталога (добавление групп, учетных записей и пр.), объединены в узел "Редактирование конфигурации". Также в руководстве:

- Добавлен новый раздел 1.6. Подключение к произвольному каталогу на текущем или удаленном LDAP-сервере (стр. 11).
- В разделе 1.7.5. Создание и редактирование учетных записей пользователей (стр. 18) описано, как импортировать учетные записи из внешних источников аутентификации (стр. 19).

2.4

Новые возможности

- Добавлена возможность импорта учетных записей из домена. Для этого на панель инструментов окна просмотра учетных записей добавлена соответствующая кнопка. Также добавлена кнопка открытия журнала, в котором можно отслеживать процесс импорта. Журнал импорта пользователей можно сохранить в файл `*.csv`, `*.pdf` или `*.xlsx`.
- Добавлена возможность включения/отключения проверки дополнительных требований к криптостойкости пароля при создании и редактировании пользователя. Для этого в файле `app_settings.json` измените значение свойства `ExtendPasswordStrength` объекта `Permissions`. Файл расположен в папке:
  - `C:\ProgramData\Automiq\HMI.SecurityConfigurator` для ОС Windows;
  - `/home/<user>/Automiq/HMI.SecurityConfigurator` для ОС Linux.

Изменение

Шаблоны приложений, поставляемые вместе с дистрибутивом конфигуратора, теперь находятся в папке:

- `C:\ProgramData\Automiq\HMI.SecurityConfigurator\AppTemplates` для ОС Windows;
- `/usr/local/share/Automiq/HMI.SecurityConfigurator/AppTemplates` для ОС Linux.

Улучшения

- Поддержана возможность задания многострочных значений строкового права, в том числе с пустыми строками.
- Добавлены новые шаблоны приложений, поставляемые вместе с дистрибутивом конфигуратора.

Исправления

- При попытке удаления учетной записи активного пользователя теперь появляется уведомление о невозможности удаления, если пользователю назначено право **`Запрет удаления активного пользователя (NonDeletableWhileActive)`** со значением **`«true»`**. Ранее такую учетную запись позволялось удалить, однако фактически удаления не происходило.
- Исправлена ошибка, из-за которой создаваемой учетной записи не удавалось назначить роль.
- Устранена причина, по которой при удалении права из одного приложения удалялись права с таким же названием из других приложений.
- Для строковых прав многострочные значения из резервной копии конфигурации теперь восстанавливаются нормально. Ранее восстанавливалось только одно значение из нескольких.
- В окне создания новой учетной записи теперь отображается предупреждение о необходимости указать пароль.

Изменения документации

Редакция 1

Изменена структура руководства пользователя:

- Бывший раздел "Запуск приложения" разделен на разделы 1.3. Запуск самостоятельного приложения (стр. 7) и 1.4. Встраивание в проект (стр. 8).
- Удален раздел "Возможности приложения", его подразделы, описывающие работу администратора в конфигураторе, стали самостоятельными разделами верхнего уровня оглавления.

Также в руководстве пользователя:

- В раздел 1.7.5. Создание и редактирование учетных записей пользователей (стр. 18) добавлен подраздел, описывающий возможность импорта учетных записей из домена (стр. 19).
- Актуализирован раздел 1.7.7. Безопасность в компонентах Альфа платформы (стр. 23): упомянуты новые шаблоны приложений, описано, как использовать право шаблона **`SecurityConfigurator`** для ограничения доступа пользователей к файловой системе в приложении Alpha.HMI.SecurityConfigurator.
- Актуализирован раздел 1.8. Настройки безопасности для разграничения возможностей пользователей (стр. 26): описано назначение нового параметра `Permissions` файла `app_settings.json`.
- Актуализирована информация о системных правах в 1.9.1. Права системного приложения Alpha.Security (стр. 28).

2.3

Улучшения

- Редактирование значений прав на просмотр и редактирование конфигурации теперь заблокировано для текущего пользователя и для группы, в которой состоит текущий пользователь.
- При экспорте приложения в файл-шаблон `*.json` теперь экспортируются не только права, но и роли приложения. При импорте такого файла в конфигурацию, соответственно, добавляются не только права, но и роли.
- В окне редактирования значения права теперь отображаются его название и описание.

Исправления

- Устранена причина, по которой в окне редактирования роли не удалялись назначенные ей права.
- Тема оформления теперь применяется корректно к экземплярам конфигуратора, добавленным в проект.

2.3.1

Изменения

- Теперь при попытке использовать в именах приложений и прав спец. символы (.,+\";<>) появляется предупреждение о запрете их использования.
- В окне создания нового пользователя больше нет полей **Пароль** и **Подтверждение пароля**. Чтобы задать пароль для новой учетной записи, используйте кнопку задания пароля. Кнопка находится рядом с предупреждением **Необходимо задать пароль**. Предупреждение и кнопка отображаются, пока пароль не будет задан.

Исправление

Исправлена ошибка, из-за которой нажатие клавиши Shift в окне просмотра учетных записей приводило к возникновению множества диалоговых окон с ошибкой **Очередь остановлена**.

2.3.2

Изменение

Видоизменена кнопка задания пароля (увеличен размер и добавлена надпись).

Исправления

- Исправлена ошибка, из-за которой не отображался список ролей в окне **Выбор роли**.
- Теперь запрещено создавать учетные записи пользователей, у которых совпадают логины и отображаемые имена без учета регистра.

2.3.3

Исправление

Исправлена ошибка совместимости с Alpha.HMI.CommonLib 2.2.0.

Изменения документации

Редакция 1

Обновлена информация в разделе 1.7.2. Создание приложений и прав (стр. 14). Здесь в подразделе Экспорт и импорт приложений (стр. 15) упомянуто, что приложение сохраняется в файл-шаблон вместе с добавленными в него правами и ролями.

Редакция 2

Обновлена информация в разделе 1.7.5. Создание и редактирование учетных записей пользователей (стр. 18). Поскольку окно создания нового пользователя изменено в версии 3.0.0, здесь описано, как теперь задавать пароль для создаваемой учетной записи.

Редакция 3

Упомянута возможность выполнения некоторых команд (создание учетных записей, создание и импорт приложений и т.п.) с помощью контекстного меню.

2.2

Новая возможность
Разработана возможность ограничения доступа пользователей к файловой системе.

- Создан тип 2.3. Разрешения (стр. 35) (SecurityConfigurator_Permissions) со свойством **FileSystemAccess (Доступность файловой системы)**, значение которого регулирует доступ к системе.
- Типу 2.2. Настройки (стр. 34) (SecurityConfigurator_Settings) добавлено свойство **ExportFolder (Папка для экспорта)**.

Если у пользователя отсутствует доступ к файловой системе (FileSystemAccess = false), то в файловых диалогах будет недоступен просмотр дерева папок и создание новых папок, а экспорт будет возможен только в папку экспорта, указанную в свойстве **ExportFolder**. По умолчанию папкой для экспорта является:

- `C:\ProgramData\Automiq\HMI.SecurityConfigurator\Export` - в ОС Windows.
- `/home/<user>/Automiq/HMI.SecurityConfigurator/Export` - в ОС Linux.

Улучшения

- Пункты контекстного меню с командами на изменение конфигурации скрыты, если пользователю запрещено редактирование конфигурации.
- Теперь при назначении прав пользователю или группе в окне выбора прав не отображаются уже назначенные права.
- При создании новой роли все права приложения больше не назначаются ей автоматически. Права приложения можно добавить или удалить, как это делается для групп и пользователей.

Изменение

Исключена возможность экспорта информации в файлы `*.pdf` во всех местах, где можно выполнить экспорт:

- экспорт списка пользователей и их прав;
- экспорт таблицы "Список прав и их значений для приложения" со страницы списка приложений;

- экспорт прав открытого на редактирование приложения.

Исправления

- Исправлена ошибка, из-за которой пользователю, добавленному в группу с нулевым значением права "История паролей", нельзя было установить текущий пароль.
- Теперь при авторизации пользователя после изменения парольных политик предлагается сменить пароль, если старый пароль не подходит новым парольным политикам. Ранее в таком случае возникала ошибка.
- Устранена причина, по которой после импорта приложения в таблице прав роли отображались не сразу, а, например, только после перехода к редактированию приложения.
- При изменении размеров окна выбора прав кнопки "Добавить" и "Отмена" оставались на месте, из-за чего ими было неудобно пользоваться. Теперь кнопки перемещаются соответственно изменению размеров окна.

2.2.1

Улучшения

- Тип **`Разрешения (SecurityConfigurator_Permissions)`** теперь позволяет использовать сервер безопасности для определения прав доступа пользователя к файловой системе.
- Панель инструментов теперь оформляется в соответствии с выбранной темой.

Исправление

При использовании Alpha.HMI.WebViewer разделитель областей окна конфигуратора не позволял нормально изменять их размеры.

2.2.2

Исправление

Максимально допустимое значение для прав **`Максимальное время сессии`** и **`Максимальное время бездействия`** увеличено до 10000. При назначении прав по умолчанию устанавливается значение 1440 (минут).

Изменения документации

Редакция 1

В руководстве пользователя:

- В разделе 1.7.7. Безопасность в компонентах Альфа платформы (стр. 23) актуализирован список шаблонов приложений Alpha.Security, поставляемых с дистрибутивом конфигуратора.
- Исключено упоминание возможности экспорта данных в файлы `*.pdf`.
- Обновлены скриншоты.

В справочном руководстве описаны новые компоненты 2.2. Настройки (стр. 34) и 2.3. Разрешения (стр. 35), а также их свойства. Для компонентов 2.1. SecurityConfigurator (стр. 33), 2.6. SecurityConfigurator_Form (стр. 37) и 2.5. SecurityConfigurator_Page (стр. 36) описаны новые свойства - ссылки на экземпляры типов 2.2. Настройки (стр. 34) и 2.3. Разрешения (стр. 35).

2.1

Новые возможности

- Внешний модуль **SecurityConfigurator** предоставляет новые компоненты для разработки проектов Alpha.HMI:
  - тип **SecurityConfigurator**, предназначенный для размещения на форме;
  - форма **SecurityConfigurator_Page** - аналог прежней формы **SecurityConfigurator**, предназначенный для открытия во фрейме;
  - форма **SecurityConfigurator_Form** - аналог прежней формы **SecurityConfigurator**, предназначенный для открытия в виде отдельного окна;
  - форма **SecurityConfigurator_App** - основная форма, предназначенная для запуска в виде отдельного приложения;
  - тип **SecurityConfigurator_Settings**, предназначенный для упрощения настройки экземпляров конфигуратора, создаваемых на основе указанных выше типов и форм.
- Для типа **SecurityConfigurator** и форм **SecurityConfigurator_Page**, **SecurityConfigurator_Form** можно:
  - выбрать тему оформления.
  - настроить вызов экранной клавиатуры в файловых диалогах.
- Реализована возможность экспорта и импорта собственных шаблонов приложений.

Улучшения

- Настройки самостоятельного приложения Alpha.HMI.SecurityConfigurator теперь сохраняются в конфигурационный файл `app_settings.json`, расположенный в:
  - `C:\ProgramData\Automiq\HMI.SecurityConfigurator` (ОС Windows);
  - `/home/<user>/Automiq/HMI.SecurityConfigurator` (ОС Linux).
- Перед удалением учетной записи пользователя теперь проверяется его активность на удаленных рабочих станциях, указанных в настройках экземпляра конфигуратора.
- Всплывающая подсказка с текстом ошибки теперь отображается всегда, без необходимости наведения мыши на элемент с ошибкой:
  - в окне редактирования учетной записи пользователя;
  - в окне смены пароля учетной записи пользователя.
- В окне просмотра учетных записей теперь можно создавать, редактировать и удалять учетные записи с помощью клавиатуры или контекстного меню. Аналогично в окне просмотра приложений можно создавать, редактировать и удалять приложения с помощью клавиатуры или контекстного меню.
- В окне просмотра групп вызвать контекстное меню для создания новой группы теперь можно без выделения существующей группы.
- Добавлены шаблоны приложений безопасности для решений Alpha.HMI.SetPoints и Alpha.HMI.IntegrityControl.
- Обновлен шаблон приложения безопасности для Alpha.HMI.Trends.
- В интерфейсе конфигуратора обновлены иконки.

2.1.1

Исправления

- Исправлены ошибки, возникавшие при создании новых учетных записей:
  - Не удавалось сохранить новую учетную запись, если при ее создании поле **Пароль** заполнялось в последнюю очередь.
  - Не появлялась всплывающая подсказка о несовпадении значений из полей **Пароль** и **Подтверждение**.
- Устранена причина повторного входа в подсистему безопасности при использовании клавиши **Enter** в окне авторизации пользователя.

2.1.2

Новая возможность

Теперь при экспорте списка пользователей в файл можно дополнительно создать файлы с перечнем эффективных прав каждого пользователя.

Исправление

Устранена причина, по которой использование клавиши Enter после ввода названия новой группы приводило к ошибке.

Изменения документации

Редакция 2

В руководстве пользователя:

- Актуализирован раздел 1.3. Запуск самостоятельного приложения (стр. 7).
- Раздел **Хранение конфигурационных файлов** переименован в 1.8. Настройки безопасности для разграничения возможностей пользователей (стр. 26) и расширен.
- Раздел **Безопасность в приложениях** переименован в 1.7.7. Безопасность в компонентах Альфа платформы (стр. 23) и актуализирован.
- Обновлены скриншоты.

Актуализировано справочное руководство.

Редакция 3

В руководстве пользователя:

- В разделе 1.7.2. Создание приложений и прав (стр. 14) создан подраздел, описывающий экспорт и импорт приложений (стр. 15).
- В разделе 1.7.5. Создание и редактирование учетных записей пользователей (стр. 18) создан подраздел, описывающий экспорт списка пользователей и списков их эффективных прав (стр. 21).
- Имя администратора Alpha.Security по умолчанию изменено с **«AdminQA»** на **«administrator»**. Имя администратора было обновлено в Alpha.Security 1.7.9.
- Обновлены скриншоты.

В истории изменений исправлен ошибочно указанный номер предыдущей редакции. Вместо "Редакция 1" указано "Редакция 2".

2.0

Важно. Текущая версия предназначена для использования с Alpha.HMI 2.0.

Изменения документации

Редакция 1

Внутренние изменения. Содержимое документа не изменилось.

1.3

Новая возможность
Списки приложений и пользователей теперь можно экспортировать в файлы форматов `*.csv`, `*.xlsx`, `*.pdf`.
Примечание: свойство **Путь для экспорта таблиц** формы **`SecurityConfigurator`**, разрабатывавшееся для реализации этой возможности, удалено из интерфейса, так как более неактуально.
Улучшения

- В окне просмотра списка приложений появилась таблица предварительного просмотра приложений. Теперь при выборе приложения из списка в области справа отображается информация о правах приложения, их значениях и назначениях.
- В окне просмотра списка пользователей теперь отображается больше информации о пользователях. Помимо логина и отображаемого имени в таблице появились номера телефонов, должности, роли и пр. Набор столбцов таблицы можно менять.

Изменения документации

Редакция 1

В руководстве пользователя описано, как экспортировать списки приложений (стр. 15) и пользователей (стр. 21) в файлы форматов `*.csv`, `*.xlsx`, `*.pdf`.

Из справочного руководства удалено упоминание устаревшего свойства **Путь для экспорта таблиц** формы **`SecurityConfigurator`**.

Во всем документе актуализированы скриншоты.

Редакция 2

Информация о конфигурационных файлах приложения актуализирована и перенесена из подраздела 1.3. Запуск самостоятельного приложения (стр. 7) -> "Настройки вида приложения" в новый раздел - 1.8. Настройки безопасности для разграничения возможностей пользователей (стр. 26).

1.2

Важно. Текущая версия предназначена для использования с Alpha.HMI 1.10.
Улучшения:

- В окне редактирования групп пользователей у дерева групп появилось контекстное меню с командами:

- **Добавить группу**;
- **Редактировать группу**;
- **Удалить группу**.

- При перезапуске конфигуратора теперь сохраняется больше параметров внешнего вида приложения.

Исправленные ошибки:

- При сортировке ролей в окне редактирования ролей значения прав отображались некорректно.
- При использовании Alpha.HMI.WebViewer в окне редактирования ролей значения строковых прав отображались некорректно.

1.2.1

Улучшение

В наборе компонентов, предоставляемых внешним модулем **SecurityConfigurator**, появилась новая экранная форма - **SecurityConfigurator_Tests**. Форма предназначена для ознакомления с примером встраивания конфигуратора безопасности в проект Alpha.HMI - здесь окно конфигуратора открывается по нажатию на кнопку **Настройка пользователей**.

Исправления

- Иконка права **PasswordAge (Срок действия пароля, дней)** в окне выбора прав отображалась некорректно при использовании приложения на ОС Linux.
- Устранена причина, по которой в окне редактирования учетных записей не сохранялась установленная вручную ширина столбцов.

Изменения документации

Редакция 1

В руководстве пользователя из раздела 1.2. Подготовка к работе (стр. 6) бывший подраздел 1.3. Запуск самостоятельного приложения (стр. 7) вынесен в самостоятельный раздел. В нем более подробно описан процесс встраивания и обновления расширения в проекте Alpha.HMI.

Редакция 2

Внутренние изменения. Содержимое документа не изменилось.

Редакция 3

В руководстве пользователя:

- В разделе 1.3. Запуск самостоятельного приложения (стр. 7) описано назначение новой экранной формы расширения - **SecurityConfigurator_Tests**.
- Обновлен раздел 1.7.6. Вычисление эффективных значений прав (стр. 22). Пример, приведенный в разделе, описан короче и проще.
- Актуализированы скриншоты.

1.1

Важно. Текущая версия предназначена для использования в Alpha.HMI 1.9.
Добавлена возможность запуска приложения по ярлыку из меню Пуск.

1.1.1

Улучшения:

- Дерево групп теперь отображает иерархию групп.
- При назначении роли или группы в окнах **Выбор группы** и **Выбор роли** было неудобно просматривать длинные названия. Окна увеличены, и их размеры можно менять для удобства.

Исправления:

- Исправлена ошибка, из-за которой пользователям и группам не назначались роли.
- Устранена причина, по которой группу нельзя было редактировать или удалить сразу после создания.
- В окне редактирования пользователя или группы при назначении роли больше не блокируется кнопка **Сохранить изменения**.
- В окне редактирования пользователя или группы при назначении роли название назначаемой роли отображалось только после нажатия кнопки **Сохранить изменения**, теперь - сразу при назначении.
- Исправлено блокирование кнопки **Добавить**/**Удалить** в окне выбора прав, происходящее после отмены выбора некоторых пунктов даже при наличии других выбранных пунктов.

Изменения документации

Редакция 1

Руководство пользователя:

- Обновлено описание запуска приложения (стр. 7).
- Обновлено описание настроек вида приложения (стр. 26).
- Актуализированы скриншоты.

Редакция 2

Актуализированы скриншоты в разделе 1.7.8. Резервное копирование конфигурации (стр. 25).

Программный комплекс Альфа платформа

Alpha.HMI.SetPoints 2.5

Руководство пользователя

Редакция	Соответствует версии ПО
1	2.5.0

© АО «Атомик Софт», 2015-2026. Все права защищены.

Авторские права на данный документ принадлежат АО «Атомик Софт». Копирование, перепечатка и публикация любой части или всего документа не допускается без письменного разрешения правообладателя.

Содержание

1. Руководство пользователя 5
1.1. О продукте 5
1.2. Подготовка к работе 5
1.3. Запуск в виде самостоятельного приложения 6
1.4. Встраивание в проект 8
1.5. Создание конфигурационного файла 9
1.6. Загрузка карт уставок 24
1.7. Чтение уставок из ПЛК 25
1.8. Редактирование значений уставок 26
1.9. Просмотр и изменение настроек уставок 28
1.10. Сохранение уставок в ПЛК 29
1.11. Сохранение в архив и просмотр архивных значений 29
1.12. Экспорт карт уставок в файл 31
1.13. Печать карт уставок 32
1.14. Настройки безопасности для разграничения возможностей пользователей 32

2. Справочное руководство 35
2.1. SetPoints 35
2.1.1. Свойства 39
2.1.2. Функции 41
2.1.3. Команды 42
2.2. SetPoints_Settings 44
2.2.1. Свойства 45
2.3. Разрешения 46
2.3.1. Свойства 46
2.4. Тема оформления Уставок 48
2.4.1. Свойства 50
2.4.2. Функции 52
2.5. SetPoints_App 54
2.6. SetPoints_Page 54
2.6.1. Свойства 54
2.7. SetPoints_Form 56
2.7.1. Свойства 56
2.8. SetPoints_Tests 58

История изменений 59
2.5 59
Изменения документации 59
Редакция 1 59
2.4 59
2.4.1 60
2.4.2 60
Изменения документации 60
Редакция 1 60
2.3 61
Изменения документации 61
Редакция 1 61
2.2 61
2.2.1 62
2.2.2 62
2.2.3 63
Изменения документации 63
Редакция 1 63

Редакция 2 64
2.1 64
2.1.1 64
Изменения документации 65
Редакция 1 65
Редакция 2 65
2.0 65
2.0.1 66
Изменения документации 66
Редакция 1 66
1.3 66
1.3.1 66
1.3.2 67
1.3.3 67
Изменения документации 67
Редакция 1 67
Редакция 2 67
1.2 67
1.2.1 68
Изменения документации 68
Редакция 1 68
Редакция 2 69
1.1 69
1.1.1 69
1.1.2 69
Изменения документации 70
Редакция 1 70
Редакция 2 70